REPUBLIQUE DU CAMEROUN
*PAIX - TRAVAIL – PATRIE*
*********
UNIVERSITE DE DSCHANG
*********
ÉCOLE DOCTORALE

REPUBLIC OF CAMEROON
*PEACE - WORK - FATHERLAND*
*********
UNIVERSITY OF DSCHANG
*********
POST GRADUATE SCHOOL

DSCHANG SCHOOL OF SCIENCES AND TECHNOLOGY

UNITE DE RECHERCHE EN INFORMATIQUE FONDAMENTALE, INGENIERIE ET APPLICATIONS (URIFIA)

*SUJET* :

# ARCHITECTURE ET MÉTHODES D'UN FRAMEWORK DE CRÉATION D'APPLICATIONS WEB PILOTÉES PAR DES MODÈLES BPMN

*Mémoire soutenu publiquement en vue de l'obtention du*

***Diplôme de Master en Informatique***

***Option*** : Informatique Fondamentale
***Spécialité*** : Intelligence Artificielle

*Par :*

**NGONG KOUT YURI ROOSEVELT**

*Matricule :* ***CM-UDS-24SCI1029***

***Licence en Informatique Fondamentale***

***Sous la direction de :***

ZEKENG NDADJI Milliam Maxime
***Chargé de Cours, Université de Dschang***

**Année académique 2025/2026**

# DÉCLARATION D'AUTHENTICITÉ

Je, soussigné NGONG KOUT Yuri Roosevelt, étudiant au Département de Mathématiques et Informatique, Faculté des Sciences, Université de Dschang, certifie que ce mémoire de Master intitulé "**ARCHITECTURE ET MÉTHODES POUR UN FRAMEWORK DE CRÉATION D'APPLICATIONS WEB PILOTÉES PAR DES MODÈLES BPMN**", réalisé au sein de l'Unité de Recherche en Informatique Fondamentale, Ingénierie et Applications (URIFIA) sous la supervision du Dr. ZEKENG NDADJI Milliam Maxime, est original. Il n'a jamais été publié auparavant pour l'obtention d'un quelconque diplôme.

*L'étudiant*
NGONG KOUT Yuri Roosevelt

.....

*Le directeur de mémoire*
ZEKENG NDADJI Milliam Maxime
*Chargé de Cours, Université de Dschang*

.....

*Le chef de département*

.....

# DÉDICACES

À Monsieur **KOUT MBEPA GEORGES** et Madame
**NGUIONG NGONG ALINE PRISCILLE**, pour leur amour,
leur soutien et leurs sacrifices.

# REMERCIEMENTS

Tout d'abord, je remercie infiniment *Dieu Tout-Puissant*, pour ses bénédictions, sa grâce et son infinie bonté. Ensuite je tiens à adresser mes sincères remerciements :

— Au **Dr. ZEKENG NDADJI Milliam Maxime**, mon encadreur académique, pour la confiance qu'il m'a témoigné en acceptant la direction scientifique de mes travaux. Je lui suis reconnaissant de m'avoir fait bénéficier tout au long de ce travail de sa rigueur intellectuelle, de sa disponibilité, de sa bienveillance, de son inspiration constante et ses précieux conseils. Votre expertise et votre soutien indéfectible ont été des piliers essentiels dans l'élaboration de ce travail ;

— À tous les **enseignants du département de Mathématiques-Informatique de la Faculté des Sciences de l'Université de Dschang**, pour la qualité de leurs enseignements et leurs précieux conseils.

— Aux distingués membres du jury qui ont accepté d'examiner ce travail. L'attention que vous porterez à cette recherche et vos commentaires critiques constitueront un enrichissement significatif pour ma formation ;

— À mes frères et sœurs, **FOSSI Benson, MBEPA Nathanaël, MOLLO Josue, KOUT Sterve, GOUA Dorcas, MANGHA Déograce, KENGNE Godwin** , merci pour votre soutien inconditionnel et votre amour ;

— Aux familles **MBEPA, NGONG** pour leur accompagnement et leurs conseils ;

— À mes amis précieux, comme **Aubin, Triomphant, Marie, Vergez, Jaures, Fomo, Dorine, Dominique, Warren, Soulemane** et tout ceux que je n'ai pas pu citer qui ont partagé avec moi cette aventure académique.

— À mes aînés académiques, **Daniella et Dimitri**, pour leur soutien inestimable et leurs conseils éclairés, qui ont grandement enrichi mon parcours et inspiré ma quête de connaissance.

— À tous ceux dont les noms ne figurent pas sur cette page ne se sentent point lésés. Votre contribution est également précieuse et grandement appréciée.

# TABLE DES MATIÈRES


Dédicaces . . . . . ii
Remerciements . . . . . iii
Table des Matières . . . . . iv
Liste des acronymes . . . . . viii
Liste des tableaux . . . . . ix
Table des figures . . . . . x
Résumé . . . . . xii
Abstract . . . . . xiii

**Introduction Générale** **1**
Contexte du travail . . . . . 1
Problématique et objectifs . . . . . 2
Organisation du manuscrit . . . . . 3

*Chapitre 1* ▸ **Généralités sur le Business Process Model and Notation et applications web métier** **5**
I.1 - Introduction . . . . . 5
I.2 - Processus métiers et gestion des processus métiers (BPM) . . . . . 5
I.2.1 - Processus métier . . . . . 6
I.2.2 - Gestion des processus métier (BPM) . . . . . 7
I.2.2.1 - Définitions de la gestion des processus métier . . . . 7
I.2.3 - Cycle de vie du BPM . . . . . 7
I.2.3.1 - Identification du processus *(Process Identification)* . . 8
I.2.3.2 - Découverte des processus *(Process Discovery)* . . . . 8
I.2.3.3 - Analyse du processus *(Process Analysis)* . . . . . 8
I.2.3.4 - Refonte du processus *(Process Redesign)* . . . . . 8
I.2.3.5 - Mise en œuvre du processus *(Process Implementation)* 8
I.2.3.6 - Suivi du processus *(Process Monitoring)* . . . . . 9
I.2.4 - Systèmes de gestion des processus métiers . . . . . 9
I.3 - Modélisation des processus métiers . . . . . 10
I.3.1 - Business Process Model and Notation (BPMN) . . . . . 10
I.3.1.1 - Définition et objectif . . . . . 10
I.3.1.2 - Historique du BPMN . . . . . 11

I.3.1.3 - Éléments de base de BPMN . . . . . 11
I.3.1.4 - Types de diagrammes BPMN . . . . . 16
I.4 - Ingénierie dirigée par les modèles . . . . . 17
I.4.1 - Principe général de l'ingénierie dirigée par les modèles . . . . 17
I.4.2 - Notion de base en Ingénierie Dirigée par les Modèles . . . . . 18
I.4.3 - Transformation de modèles . . . . . 19
I.4.4 - Model-Driven Architecture . . . . . 19
I.5 - Applications web métier . . . . . 20
I.5.1 - Généralités sur le développement web . . . . . 20
I.5.1.1 - Historique du développement web . . . . . 20
I.5.1.2 - Définition d'application web et concepts du web . . . 21
I.5.1.3 - Frameworks d'application web . . . . . 22
I.5.1.4 - Sécurité dans les applications web . . . . . 23
I.5.2 - Applications web métier . . . . . 24
I.5.2.1 - Définitions et principe . . . . . 24
I.5.2.2 - Moteur de processus . . . . . 24
I.6 - Conclusion . . . . . 25

*Chapitre II* ▸ **État de l'art sur la génération d'applications web pilotées par des modèles** **26**
II.1 - Introduction . . . . . 26
II.2 - Démarche d'analyse des travaux existants . . . . . 26
II.3 - Revue de la littérature . . . . . 27
II.3.1 - Systèmes d'information orientés processus et plateformes d'exécution . . . . . 27
II.3.1.1 - Moteurs BPMS et plateformes d'exécution . . . . . 28
II.3.1.2 - Architecture web pilotée par BPMN . . . . . 28
II.3.1.3 - WebWorkFlow . . . . . 29
II.3.1.4 - Plateforme AKIP . . . . . 30
II.3.2 - Dérivation d'artefacts applicatifs à partir de BPMN . . . . . 30
II.3.2.1 - Transformation vers des modèles d'interaction . . . . 31
II.3.2.2 - Génération de prototypes et d'API . . . . . 32
II.3.2.3 - Vers une génération intégrée des artefacts applicatifs 32
II.3.3 - Ingénierie dirigée par les modèles et génération de systèmes orientés processus . . . . . 34
II.3.3.1 - Composition de langages et génération progressive . 34
II.3.3.2 - Gabarits et compléments aux modèles BPMN . . . . 35

II.3.4 - Maintenabilité du code généré et mise en perspective avec le low-code . . . 35
II.3.4.1 - Gestion conjointe du code généré et du code manuel 36
II.3.4.2 - Mise en perspective avec les approches low-code . . 36
II.4 - Discussion et positionnement de la recherche . . . 36
II.5 - Conclusion . . . 38

*Chapitre III* ▶ **Approche d'ingénierie proposée pour la génération d'applications web pilotées par BPMN** **39**
III.1 - Introduction . . . 39
III.2 - Vue d'ensemble de l'approche . . . 40
III.2.1 - Problème traité et principe général . . . 40
III.2.2 - Positionnement dans le cycle MDA . . . 40
III.2.3 - Contribution méthodologique de l'approche . . . 41
III.3 - Architecture globale du framework proposé . . . 41
III.3.1 - Modèles d'entrée de ProOWeb . . . 42
III.3.2 - Noyau de génération du framework . . . 42
III.3.3 - Application web générée . . . 43
III.3.4 - Exploitation et évolution de l'application . . . 43
III.4 - Modèle d'entrée enrichi : spécification spec-v1 . . . 43
III.4.1 - Structure générale de spec-v1 . . . 44
III.4.2 - Métadonnées des activités . . . 45
III.4.3 - Affectation des tâches et sécurité . . . 46
III.4.4 - Entrées, sorties et données partagées . . . 46
III.4.5 - Règles de visibilité . . . 47
III.5 - Règles de transformation et génération de l'application web . . . 47
III.5.1 - Validation avant génération . . . 48
III.5.2 - Transformation modèle-à-modèle . . . 49
III.5.3 - Transformation modèle-à-texte . . . 50
III.5.4 - Exemple de traçabilité de bout en bout . . . 50
III.5.5 - Règles de projection vers les artefacts . . . 51
III.6 - Préservation du code personnalisé et synchronisation . . . 52
III.6.1 - Problème de la régénération . . . 52
III.6.2 - Manifeste et contrat de propriété . . . 52
III.6.3 - Préservation côté logique applicative . . . 53
III.6.4 - Préservation côté interface utilisateur . . . 53
III.6.5 - Cycle de synchronisation itératif . . . 53

III.7 - Conclusion . . . . . 54

*Chapitre IV* ▸ **Mise en oeuvre et évaluation de ProOWeb** **55**
IV.1 - Introduction . . . . . 55
IV.2 - Environnement de mise en oeuvre . . . . . 56
IV.2.1 - Objectif de réalisation . . . . . 56
IV.2.2 - Choix technologiques . . . . . 56
IV.2.3 - Organisation réalisée . . . . . 57
IV.3 - Implémentation de la chaîne de génération . . . . . 58
IV.3.1 - Initialisation du projet cible . . . . . 59
IV.3.2 - Gestion des modèles de processus . . . . . 59
IV.3.3 - Validation et construction des contrats . . . . . 60
IV.3.4 - Génération des artefacts applicatifs . . . . . 60
IV.3.5 - Préservation et régénération contrôlée . . . . . 61
IV.4 - Cas d'étude organisationnel . . . . . 62
IV.4.1 - Présentation de l'organisation retenue . . . . . 62
IV.4.2 - Acteurs et responsabilités . . . . . 63
IV.4.3 - Processus retenus . . . . . 63
IV.4.4 - Déroulement des scénarios . . . . . 64
IV.4.5 - Spécifications enrichies associées . . . . . 65
IV.4.6 - Application web métier générée . . . . . 66
IV.5 - Évaluation de ProOWeb . . . . . 68
IV.5.1 - Méthode d'évaluation . . . . . 68
IV.5.2 - Résultats observés . . . . . 69
IV.5.3 - Analyse de la capacité multi-processus . . . . . 71
IV.5.4 - Apports observés . . . . . 72
IV.5.5 - Limites observées . . . . . 72
IV.5.6 - Discussion . . . . . 73
IV.6 - Conclusion . . . . . 73

**Conclusion générale** **74**
Bilan du travail . . . . . 74
Quelques perspectives . . . . . 75

**Bibliographie** **76**

# LISTE DES ACRONYMES

| | |
|---|---|
| AFNOR | Association française de normalisation ; |
| AKIP | AKIP Process Automation Platform ; |
| API | Application Programming Interface ; |
| ATL | ATLAS Transformation Language ; |
| BPM | Business Process Management ; |
| BPMI | Business Process Management Institute ; |
| BPMN | Business Process Model and Notation ; |
| BPMS | Business Process Management System ; |
| CIM | Computation Independent Model ; |
| DMN | Decision Model and Notation ; |
| EPC | Event-driven Process Chain ; |
| HTML | HyperText Markup Language ; |
| HTTP | HyperText Transfer Protocol ; |
| IBM | International Business Machines ; |
| IDM | Ingénierie dirigée par les modèles ; |
| IFML | Interaction Flow Modeling Language ; |
| jBPM | Java Business Process Management ; |
| JSON | JavaScript Object Notation ; |
| M2M | Model-to-Model ; |
| M2T | Model-to-Text ; |
| MDA | Model Driven Architecture ; |
| MOF | Meta-Object Facility ; |
| OMG | Object Management Group ; |
| PIM | Platform Independent Model ; |
| PSM | Platform Specific Model ; |
| QVT | Query/View/Transformation ; |
| REST | Representational State Transfer ; |
| RFC | Request for Comments ; |
| SPA | Single Page Application ; |
| SWEL | Domain-Specific Language for Modeling Data-Intensive Workflows ; |
| UML | Unified Modeling Language ; |
| WebDSL | Web Domain-Specific Language ; |
| XOR | Exclusive OR. |

# LISTE DES TABLEAUX

I - Composantes essentielles d'un processus métier . . . . . 6

II - Grille concrète de lecture des travaux étudiés . . . . . 27
III - Comparaison de travaux représentatifs étudiés . . . . . 37

IV - Positionnement de ProOWeb dans le cycle MDA . . . . . 41
V - Définition formelle des principaux champs de la spécification *spec-v1* . 45
VI - Exemples de stratégies d'affectation . . . . . 46
VII - Correspondance entre informations d'entrée et application générée . . 47
VIII - Exemples de contrôles réalisés avant génération . . . . . 48
IX - Structures principales du modèle pivot ProOWeb . . . . . 49
X - Exemple de traçabilité d'une activité BPMN vers les artefacts générés . 50
XI - Exemples de règles de projection vers l'application générée . . . . . 51
XII - Inventaire des artefacts générés par ProOWeb . . . . . 51
XIII - Mécanismes de préservation et de synchronisation dans ProOWeb . . 54

XIV - Principaux choix de mise en oeuvre de ProOWeb . . . . . 57
XV - Organisation fonctionnelle réalisée dans ProOWeb . . . . . 58
XVI - Contrats produits avant la génération des fichiers . . . . . 60
XVII - Acteurs du cas d'étude e-commerce . . . . . 63
XVIII - Processus métier retenus dans le cas d'étude . . . . . 64
XIX - Scénarios exécutés dans le cas pratique . . . . . 65
XX - Synthèse des spécifications enrichies par processus . . . . . 66
XXI - Critères retenus pour l'évaluation . . . . . 69
XXII - Synthèse des résultats observés sur le cas d'étude . . . . . 70
XXIII - Parcours vérifiés par les tests Cypress . . . . . 71

# TABLE DES FIGURES

1 - Exemple de processus métier . . . . . 6
2 - Cycle de vie du BPM [12] . . . . . 7
3 - Historique du BPMN [15] . . . . . 11
4 - Les différents éléments de BPMN 2.0. . . . . 12
5 - Les types des tâches dans BPMN 2.0. . . . . 13
6 - Événements dans BPMN 2.0. . . . . 13
7 - Les branchements de BPMN 2.0. . . . . 14
8 - Les flux de séquence de BPMN 2.0. . . . . 14
9 - La représentation d'un pool avec deux lanes en BPMN 2.0. . . . . 15
10 - Les données de BPMN 2.0. . . . . 15
11 - Les artéfacts. . . . . 16
12 - Exemple de diagramme de processus . . . . . 16
13 - Exemple de diagramme de collaboration . . . . . 17
14 - Exemple de diagramme de chorégraphie . . . . . 17
15 - Pyramide de modélisation de l'OMG.[5]. . . . . 18
16 - Schéma de base d'une transformation de modèle [16] . . . . . 19
17 - Illustration des trois niveaux d'abstraction du MDA [16] . . . . . 20
18 - Coeur du modèle RBAC [26] . . . . . 24
19 - Modélisation d'un processus BPMN exécutable dans Camunda Modeler, d'après la documentation Camunda [27]. . . . . 28
20 - Fonctionnalités et technologies d'une application web pilotée par processus [35]. . . . . 29
21 - Interface générée dans WebWorkFlow pour une tâche de workflow, extraite de [36]. . . . . 29
22 - Architecture et principe de génération d'une application dans AKIP, adaptés de [37]. . . . . 30
23 - Transformation d'un modèle IFML vers un modèle Symfony, adaptée de [38]. . . . . 31
24 - Cycle de prototypage rapide d'applications web proposé par [41]. . . . 33
25 - Extraits de code généré pour le contrôleur et la vue dans l'approche de [41]. . . . . 33
26 - Génération en deux étapes d'un système d'information orienté processus[9]. 34

27 - Architecture générale de ProOWeb . . . . . 42

28 - Chaîne de génération de ProOWeb . . . . . 58
29 - Initialisation du projet `E-commerce_Application` . . . . . 59
30 - Portail de connexion à l'application `E-commerce_Application` . . . 67
31 - Tableau de bord pour un utilisateur connecté avec le rôle `CLIENT` . . . 67
32 - Tableau de bord pour un utilisateur connecté avec le rôle `GESTIONNAIRE_STOCK` 68
33 - Tableau de bord pour un utilisateur connecté avec le rôle `SERVICE_COMMERCIAL` 68

# RÉSUMÉ

Ce mémoire s'intéresse à la génération d'applications web métier à partir de modèles de processus. Il part du constat que, dans de nombreuses organisations, les processus décrivent déjà une part essentielle du travail à informatiser : les activités à enchaîner, les acteurs concernés, les informations manipulées et les décisions à prendre. Pourtant, au moment de développer une application web, ces modèles restent souvent cantonnés à un rôle documentaire et leur contenu doit être réinterprété manuellement par les développeurs. Cette séparation fragilise la cohérence entre le besoin métier et le logiciel produit, complique la maintenance et rend l'évolution des applications plus coûteuse lorsque les procédures changent. L'objectif de ce travail est donc de concevoir un framework capable d'exploiter plus directement un modèle BPMN enrichi pour produire une application web métier structurée, traçable et adaptable. Pour cela, nous avons conçu ProOWeb, un framework qui associe le modèle BPMN, chargé de représenter le déroulement du processus, à une spécification complémentaire. Cette spécification apporte les informations que le modèle ne porte pas suffisamment, notamment les rôles, les affectations, les données, les entrées, les sorties, les règles de visibilité et les indications nécessaires à la génération des formulaires. Le framework valide ces éléments, les organise dans un modèle pivot, puis les projette vers les artefacts d'une application web métier : vues de traitement, logique applicative, structures de données, règles d'accès, suivi des instances et traces de génération. Le travail met également l'accent sur la régénération contrôlée, afin que l'application produite puisse évoluer sans rompre le lien entre le processus, la spécification et le code. Ainsi, ProOWeb ne vise pas seulement à automatiser l'écriture de fichiers, mais à rapprocher durablement la modélisation métier et la réalisation logicielle.

**Mots clés :** Processus métier, application web métier, BPMN, ProOWeb, génération d'application, ingénierie dirigée par les modèles, spécification enrichie, régénération contrôlée.

# ABSTRACT

This thesis focuses on generating business web applications from process models. It begins with the observation that, in many organizations, processes already describe a significant portion of the work to be computerized : the activities to be sequenced, the stakeholders involved, the information handled, and the decisions to be made. However, when developing a web application, these models often remain confined to a purely documentary role, and their content must be manually reinterpreted by developers. This separation weakens the coherence between business needs and the software produced, complicates maintenance, and makes application evolution more expensive when procedures change. The objective of this work is therefore to design a framework capable of making more direct use of an enriched BPMN model to produce a structured, traceable, and adaptable business web application. To this end, we have designed ProOWeb, a framework that combines the BPMN model, responsible for representing the process flow, with a complementary specification. This specification provides the information that the model does not adequately contain, including roles, assignments, data, inputs, outputs, visibility rules, and the necessary information for form generation. The framework validates these elements, organizes them into a pivot model, and then projects them onto the artifacts of a business web application : processing views, application logic, data structures, access rules, instance tracking, and generation traces. The work also emphasizes controlled regeneration, ensuring that the resulting application can evolve without breaking the link between the process, the specification, and the code. Thus, ProOWeb aims not only to automate file writing but also to foster a lasting connection between business modeling and software development.

**Keywords :** Business process, business web application, BPMN, ProOWeb, application generation, model-driven engineering, rich specification, controlled regeneration.

# INTRODUCTION GÉNÉRALE

SOMMAIRE

Contexte du travail . . . . . 1
Problématique et objectifs . . . . . 2
Organisation du manuscrit . . . . . 3

## Contexte du travail

Une part importante des services proposés aujourd'hui sur le web repose sur des activités organisées sous forme de processus métier. Lorsqu'un usager effectue une inscription en ligne, dépose une demande administrative, suit une réclamation, réalise une commande ou valide une opération interne dans une organisation, il participe à une suite d'actions coordonnées, portées à la fois par des humains, des règles métier et des systèmes logiciels. Dans ce contexte, l'application web ne se limite plus à présenter des pages ou à stocker des informations : elle devient le support d'exécution d'un travail métier, avec des rôles, des formulaires, des données, des validations et des enchaînements d'activités.

Le Business Process Management (BPM) fournit un cadre pour identifier, modéliser, analyser et améliorer ces processus. Les travaux de référence dans ce domaine montrent que la maîtrise d'un processus passe autant par sa représentation explicite que par sa capacité à être exécuté et adapté dans le système d'information [1, 2]. Parmi les notations utilisées, BPMN occupe une place importante, car elle permet de décrire les acteurs, les événements, les tâches et les flux de contrôle d'un processus selon une notation largement reconnue [3]. Cependant, un modèle BPMN, même correct du point de vue du processus, ne suffit généralement pas à produire directement une application web métier complète. Il indique ce qui doit s'enchaîner, mais ne décrit pas toujours avec précision les écrans attendus, les données manipulées, les champs visibles ou obligatoires, les règles de création des

formulaires, ni la manière dont les utilisateurs interagissent concrètement avec le processus.

Parallèlement, l'ingénierie dirigée par les modèles propose de placer les modèles au centre du développement logiciel, afin de transformer des descriptions abstraites en artefacts plus proches de l'exécution [4, 5]. Cette orientation est particulièrement intéressante pour les applications web orientées processus, car elle permet d'envisager une continuité entre la compréhension métier, la spécification du processus et la production de l'application. Toutefois, les approches existantes restent souvent limitées à la génération de fragments d'interface, de prototypes ou de structures partielles. Plusieurs travaux ont exploré le passage de BPMN vers des modèles d'interaction ou vers des applications guidées par les processus [6–9], mais le problème demeure lorsqu'il s'agit de produire une application web métier fonctionnelle, utilisable et cohérente avec les exigences exprimées dans le modèle.

C'est dans cette perspective que s'inscrit ce mémoire. Le travail présenté porte sur ProOWeb, un framework d'ingénierie visant à générer une application web métier complète et fonctionnelle à partir de modèles BPMN enrichis. L'idée centrale n'est donc pas seulement de dessiner un processus, ni de produire une maquette, mais de faire du modèle enrichi une source exploitable pour obtenir une application capable de gérer les acteurs, les tâches, les données, les formulaires, les transitions et le suivi des instances. ProOWeb cherche ainsi à rapprocher la modélisation métier et la réalisation logicielle, tout en conservant une organisation suffisamment claire pour permettre l'évolution de l'application générée.

## Problématique et objectifs

La difficulté principale vient du décalage entre la richesse attendue d'une application web métier et les informations habituellement contenues dans un modèle BPMN standard. Un processus peut préciser qu'une tâche doit être exécutée par un acteur donné, mais l'application correspondante doit aussi savoir quelles données afficher, quelles informations demander, quelles validations appliquer, quelles pages produire, comment relier les actions de l'utilisateur à l'avancement du processus et comment préserver les parties personnalisées lorsque le modèle évolue. Sans mécanisme structuré, le passage du modèle au code devient manuel, répétitif et fragile ; il expose le projet à des incohérences entre le processus conçu et l'application réellement livrée.

La problématique de ce travail peut donc être formulée de la manière suivante : comment concevoir un framework permettant, à partir de modèles BPMN enrichis,

de générer une application web métier complète et fonctionnelle, tout en conservant la cohérence entre les processus, les données, les rôles, les interfaces et l'évolution future de l'application ?

L'objectif principal de ce mémoire est de proposer ProOWeb, un framework de génération d'applications web métier pilotées par des modèles BPMN enrichis. Ce framework vise à transformer une spécification métier en une application exploitable, dans laquelle les tâches des processus sont reliées à des écrans, à des données et à des règles d'interaction clairement définies.

De manière spécifique, il s'agit de :

— étudier les notions de processus métier, de BPMN, d'applications web orientées processus et d'ingénierie dirigée par les modèles, afin de poser les bases conceptuelles du travail ;
— analyser les travaux existants sur la génération d'applications web à partir de modèles de processus, en mettant en évidence leurs apports et leurs limites ;
— définir un modèle d'entrée enrichi permettant de compléter BPMN par les informations nécessaires à la génération d'une application web métier fonctionnelle ;
— proposer l'architecture générale de ProOWeb, ses modèles internes et ses règles de transformation ;
— montrer comment le framework peut être mis en œuvre et évalué à travers la génération d'une application web métier représentative, construite autour de plusieurs processus d'une même organisation.

Ainsi, la contribution attendue ne se limite pas à un outil de production de code. Elle porte d'abord sur une démarche d'ingénierie : préciser ce qui doit être ajouté au modèle, organiser la transformation, garantir la traçabilité entre les éléments métier et les éléments générés, puis permettre l'obtention d'une application web utilisable par les acteurs du processus.

## Organisation du manuscrit

La suite de ce mémoire est organisée en quatre chapitres, suivis d'une conclusion générale.

**Chapitre I : Généralités sur le Business Process Model and Notation et les applications web métiers.** Ce chapitre présente les concepts de base nécessaires à la compréhension du sujet. Il revient sur les processus métier, le BPM, la notation BPMN, l'ingénierie dirigée par les modèles et les applications web métier. Il

permet ainsi de fixer le vocabulaire et le cadre conceptuel utilisés dans la suite du mémoire.

**Chapitre II : Revue de la littérature sur la génération d'applications web pilotées par des modèles.** Ce chapitre étudie les travaux existants liés à l'automatisation des processus, aux systèmes BPM, à la génération de code, aux approches low-code/no-code et aux transformations de modèles. Il met en évidence les limites rencontrées lorsqu'il faut passer d'un modèle de processus à une application web métier complète, fonctionnelle et évolutive.

**Chapitre III : Framework d'ingénierie proposé pour la génération d'applications web pilotées par BPMN.** Ce chapitre constitue le cœur de la contribution. Il présente ProOWeb, son principe général, son architecture, son modèle d'entrée enrichi, ses règles de transformation et les mécanismes prévus pour produire une application web métier fonctionnelle tout en conservant la possibilité d'évolution et de personnalisation.

**Chapitre IV : Mise en œuvre et évaluation de ProOWeb.** Ce chapitre est consacré à la réalisation concrète du framework et à son évaluation. Il présente l'application générée pour une organisation de commerce électronique, les choix de mise en œuvre, les trois processus retenus, les interfaces obtenues, les résultats observés ainsi que les limites du travail réalisé.

**Conclusion générale :** elle rappelle la problématique traitée, synthétise les contributions du mémoire, discute les résultats obtenus et ouvre quelques perspectives pour l'amélioration de ProOWeb.

# I

CHAPITRE

# GÉNÉRALITÉS SUR LE BUSINESS PROCESS MODEL AND NOTATION ET APPLICATIONS WEB MÉTIER

SOMMAIRE


I.1 - Introduction . . . . . 5
I.2 - Processus métiers et gestion des processus métiers (BPM) . . . . . 5
I.3 - Modélisation des processus métiers . . . . . 10
I.4 - Ingénierie dirigée par les modèles . . . . . 17
I.5 - Applications web métier . . . . . 20
I.6 - Conclusion . . . . . 25


## I.1. Introduction

Dans ce chapitre, nous présentons les notions de base nécessaires à la compréhension de notre travail. Nous abordons dans un premier temps les processus métier, le BPM et BPMN. Dans un second temps, nous rappelons les concepts du développement web utiles à l'étude des applications métier. Cette présentation permet de préciser le lien entre modèle métier, moteur de processus et application web.

## I.2. Processus métiers et gestion des processus métiers (BPM)

### I.2.1. Processus métier

Un processus peut être compris comme un ensemble d'activités corrélées qui transforment des éléments d'entrée en éléments de sortie. Cette définition, proche de celle de l'ISO 9000, met l'accent sur l'enchaînement, la transformation et le résultat attendu. Hammer et Champy définissent également le processus comme un ensemble finalisé d'activités produisant une valeur pour un client [10].

Un processus métier est un processus lié à un objectif concret de l'organisation. Il décrit comment une structure produit un service, traite une demande, prend une décision ou coordonne plusieurs intervenants.

Le caractère métier du processus est important. Le processus exprime d'abord une réalité organisationnelle avant de devenir une réalité informatique. Il précise ce qui doit être fait, par qui, avec quelles informations, selon quelles règles et avec quel résultat.

Un processus métier comporte généralement trois dimensions. La dimension fonctionnelle répond à la question : que faut-il faire ? La dimension organisationnelle répond à la question : qui le fait ? La dimension informationnelle répond à la question : quelles données sont nécessaires ou produites ? Ces trois dimensions forment une base utile pour passer ensuite vers la conception d'un système d'information.

**TABLE I –** *Composantes essentielles d'un processus métier*

| Composante | Rôle dans le processus |
|---|---|
| Entrées | Éléments nécessaires au démarrage ou à l'exécution : demande, formulaire, document, événement ou information reçue. |
| Activités | Opérations réalisées par des acteurs humains ou des systèmes. |
| Acteurs | Personnes, services, rôles ou applications responsables des activités. |
| Règles | Conditions qui orientent l'exécution, les décisions, les validations et les autorisations. |
| Sorties | Résultats produits : dossier validé, demande rejetée, document généré, notification envoyée. |

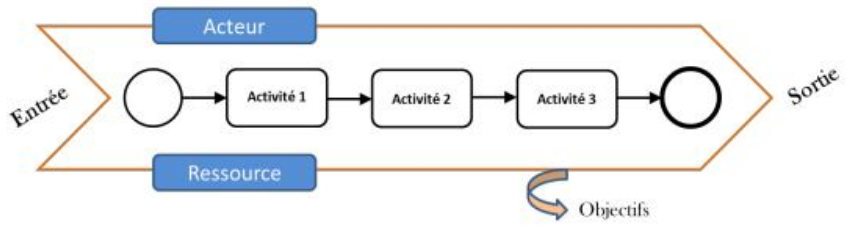



**FIGURE 1 –** *Exemple de processus métier*

## I.2.2. Gestion des processus métier (BPM)

### I.2.2.1. Définitions de la gestion des processus métier

Plusieurs définitions ont été données pour le terme de gestion des processus métier.

D'après Van Der Aalst et al. , le *Business Process Management* (BPM) comprend « les méthodes, les techniques et les outils permettant de concevoir, d'exécuter, de gérer et d'analyser les processus opérationnels, en s'appuyant sur des acteurs pouvant être des êtres humains, des organisations, des applications et d'autres sources d'information » [11].

Le *Business Process Management* (BPM) peut également être défini comme l'art de modéliser, de gérer et d'optimiser les processus métiers afin d'améliorer la performance des entreprises. Il nécessite l'utilisation d'outils permettant de modéliser, d'intégrer, de superviser et d'analyser le fonctionnement des processus tout au long de leur cycle de vie, dans une logique d'amélioration continue [1, 2].

## I.2.3. Cycle de vie du BPM

Selon Marlon Dumas [12], le BPM peut être considéré comme un cycle continu comprenant les phases suivantes

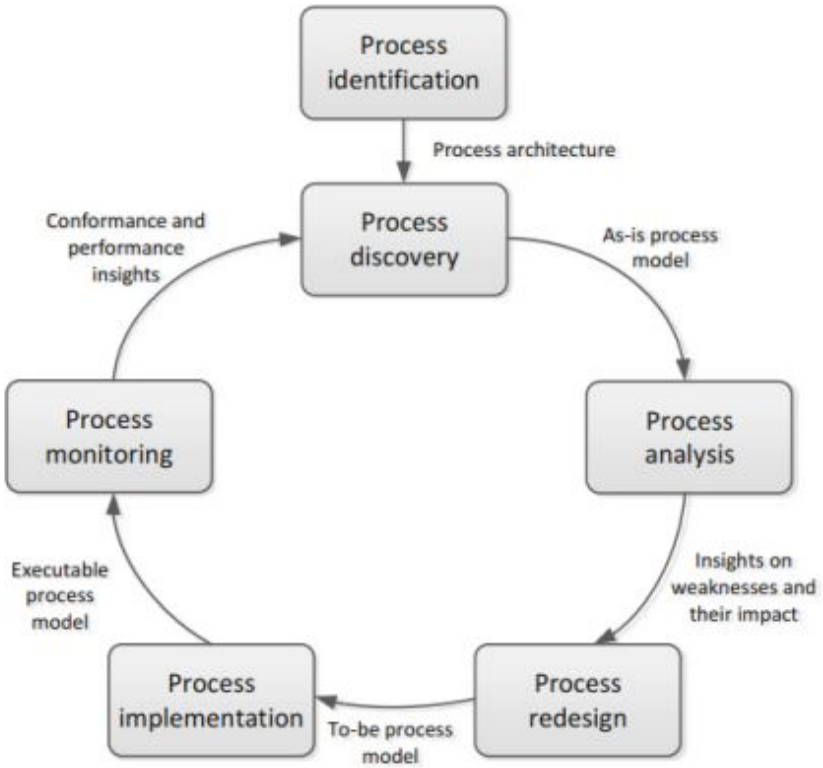



**FIGURE 2 –** *Cycle de vie du BPM [12]*

#### I.2.3.1. Identification du processus *(Process Identification)*

Cette étape part d'un problème concret rencontré dans l'entreprise. Il s'agit alors de repérer les processus concernés, d'en préciser les limites et de comprendre les liens qu'ils entretiennent avec les autres processus. Ce travail conduit à une architecture de processus, nouvelle ou actualisée, qui donne une vue d'ensemble du fonctionnement de l'organisation. Cette architecture sert ensuite de base pour choisir le ou les processus qui seront traités dans les autres phases du cycle de vie. L'identification des processus s'accompagne souvent de la définition des indicateurs de performance.

#### I.2.3.2. Découverte des processus *(Process Discovery)*

Cette phase correspond à la description de la situation existante. Elle consiste à représenter le fonctionnement réel des processus retenus, le plus souvent à travers un ou plusieurs modèles de processus. Elle permet ainsi de disposer d'une vision claire de la manière dont le travail est effectivement réalisé.

#### I.2.3.3. Analyse du processus *(Process Analysis)*

À ce niveau, les difficultés liées au processus sont relevées, décrites et, lorsque cela est possible, mesurées à l'aide d'indicateurs de performance. Le résultat obtenu est une liste organisée des problèmes constatés. Ces problèmes sont ensuite hiérarchisés selon leur importance, leur impact sur le processus et l'effort nécessaire pour les corriger.

#### I.2.3.4. Refonte du processus *(Process Redesign)*

Cette phase, aussi appelée amélioration du processus, vise à proposer les modifications capables de résoudre les problèmes identifiés précédemment et d'aider l'organisation à atteindre ses objectifs de performance. Plusieurs possibilités de changement sont étudiées, puis comparées à partir des indicateurs retenus. Ainsi, la refonte reste étroitement liée à l'analyse : chaque solution envisagée doit être évaluée avant d'être retenue. Les options les plus pertinentes sont finalement regroupées pour produire un processus amélioré, généralement présenté sous forme de modèle.

#### I.2.3.5. Mise en œuvre du processus *(Process Implementation)*

Cette étape consiste à préparer et à appliquer les changements permettant de passer du processus actuel au processus cible. Elle comporte deux dimensions principales : le changement organisationnel et l'automatisation. Le changement organisationnel concerne l'adaptation des pratiques de travail des acteurs impliqués dans

le processus. L'automatisation, quant à elle, renvoie à la mise en place ou à l'amélioration des systèmes informatiques destinés à soutenir le nouveau processus.

#### I.2.3.6. Suivi du processus *(Process Monitoring)*

Lorsque le processus amélioré est exécuté, des données sont collectées puis analysées afin d'évaluer sa performance par rapport aux objectifs fixés. Cette observation permet de détecter les lenteurs, les erreurs fréquentes ou les écarts par rapport au fonctionnement prévu. Des actions correctives peuvent alors être engagées. De nouveaux problèmes pouvant apparaître dans le même processus ou dans d'autres, le cycle BPM doit être repris de manière continue.

### I.2.4. Systèmes de gestion des processus métiers

Les systèmes de gestion des processus métier, généralement désignés par l'abréviation BPMS *(Business Process Management Systems)*, désignent des solutions logicielles conçues pour accompagner la mise en œuvre des démarches BPM. Ces environnements peuvent intégrer plusieurs fonctionnalités, notamment la modélisation des processus, l'exécution des workflows, la gestion des tâches, le suivi à travers des tableaux de bord, l'interconnexion avec d'autres applications, la supervision des activités et, dans certains cas, la conception d'interfaces utilisateur [13]. Parmis ces logiciel nous pouvons citer :

— **Camunda** : Camunda est une plateforme d'orchestration de processus qui s'appuie notamment sur les standards BPMN et DMN. Elle est souvent utilisée pour automatiser des processus complexes impliquant des utilisateurs, des applications, des services et des règles de décision. Camunda est particulièrement adaptée aux environnements techniques distribués et aux architectures orientées services.

— **Bonita / Bonitasoft** : Bonita est une plateforme BPMS orientée vers l'automatisation et l'optimisation des processus métier. Elle permet de modéliser des processus, de les exécuter, de gérer les tâches utilisateurs et d'intégrer les processus avec d'autres systèmes d'information. Son intérêt réside dans son caractère extensible et dans sa séparation entre la modélisation métier, le développement technique et l'exécution des processus.

— **IBM Business Automation Workflow** : IBM Business Automation Workflow est une plateforme d'automatisation des workflows numériques. Elle combine la gestion des processus métier et la gestion de cas afin de coordonner les utilisateurs, les informations et les activités. Elle est surtout

utilisée dans les grandes organisations ayant besoin de solutions robustes, sécurisées et intégrées à leur système d'information.

— **Flowable** : Flowable est une plateforme de gestion et d'automatisation des processus métier basée sur des standards ouverts. Elle est utilisée pour exécuter des workflows, gérer des processus complexes et coordonner des tâches humaines ou automatisées. Son intérêt se trouve dans sa flexibilité et dans sa capacité à être intégrée dans des applications d'entreprise.

## I.3. Modélisation des processus métiers

La modélisation des processus métier consiste à représenter le fonctionnement d'un processus afin de préciser les activités qui le composent ainsi que leur ordre d'exécution. Dans la littérature, deux grandes approches sont généralement distinguées pour décrire le comportement d'un processus [14] :

— **L'approche impérative** : elle décrit de manière explicite la façon dont le processus doit être exécuté. Elle précise donc l'enchaînement des activités, les conditions de passage d'une étape à une autre et les différents chemins possibles. Les langages tels que BPMN, les diagrammes d'activités UML ou encore les EPC s'inscrivent dans cette logique.

— **L'approche déclarative** : elle s'intéresse davantage aux contraintes que le processus doit respecter qu'à la description détaillée de toutes les séquences d'exécution. Autrement dit, elle indique ce qui est autorisé, obligatoire ou interdit, sans imposer un chemin unique d'exécution. Parmi les langages associés à cette approche, on peut citer Declare, ConDec, PLM-flow et PENELOPE.

### I.3.1. Business Process Model and Notation (BPMN)

#### I.3.1.1. Définition et objectif

BPMN, pour *Business Process Model and Notation*, est un standard de modélisation des processus métier maintenu par l'Object Management Group. Sa version 2.0.2 définit à la fois une notation graphique et des éléments de méta-modèle permettant de représenter des processus, des événements, des activités, des passerelles, des participants et des flux [3]. Son intérêt principal est de proposer un langage visuel compréhensible par les acteurs métier, tout en restant exploitable par des outils informatiques.

BPMN répond à un besoin fréquent dans les organisations : disposer d'une notation commune pour décrire les processus. Avant l'existence d'une notation partagée, chaque équipe pouvait représenter les processus à sa manière, avec des symboles locaux et des règles implicites. Cette diversité rendait les échanges difficiles. BPMN cherche à réduire cette ambiguïté en fournissant une grammaire visuelle plus stable.

### I.3.1.2. Historique du BPMN

Le Mark von Roding et al. ont cité l'historique de BPMN dans son livre [15] comme suit :

— En 2001, y avait de nombreuses notations et points de vue différents de modélisation, c'est dans ce cas où les membres de Business Process Management Institute (BPMI) ont commencé à discuter de l'idée de normaliser les techniques orientées pour représenter visuellement les composants de processus et aligner la notation avec un langage de processus exécutable.
— En mai 2004, c'était la publication de la première version de BPMN (BPMN 1.0).
— En 6 février 2006, BPMI (Business Process Management Institute) a été remplacée par l'OMG qui a depuis maintenu et développé la norme BPMN.
— En janvier 2008, la version BPMN 1.1 a été publiée par l'OMG.
— En 2009, la version BPMN 1.2 a été publiée par l'OMG.
— Janvier 2011, la version BPMN 2.0 a été publiée par l'OMG.
— En décembre 2013, la version 2.0.2 de BPMN a été publié par l'OMG.




**FIGURE 3 –** *Historique du BPMN [15]*

### I.3.1.3. Éléments de base de BPMN

Un diagramme BPMN est composé d'un ensemble d'éléments graphiques qui permettent de modéliser les activités, les flux, les relations, les données et l'interaction entre ces éléments.

Les différents éléments sont organisés en cinq catégories :

— les objets de flux *(Flow Objects)* ;

— les objets de connexion *(Connecting Objects)* ;
— les partitions *(Swimlanes)* ;
— les données ;
— les artéfacts *(Artifacts)*.

Les différents éléments de BPMN 2.0 sont illustrés dans la figure ci-dessous.

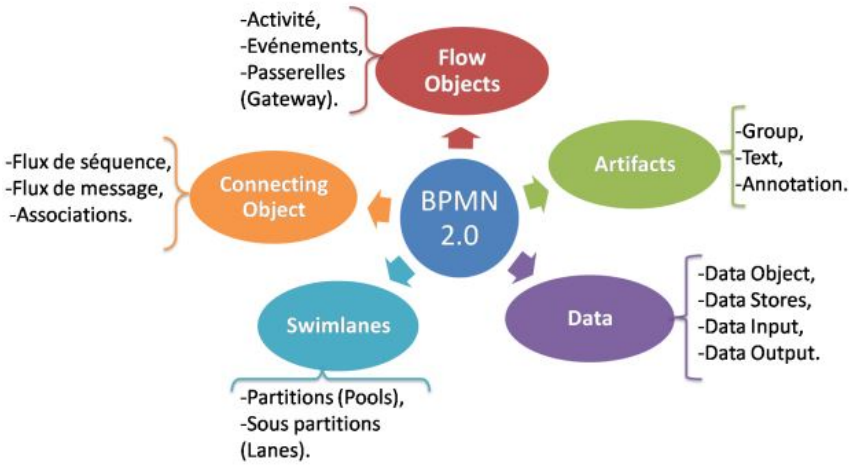



**FIGURE 4 –** *Les différents éléments de BPMN 2.0.*

### A. Les objets de flux

Les objets de flux sont les principaux éléments graphiques qui permettent de définir le comportement d'un processus métier. Il existe trois types d'objets de flux : les activités, les événements et les passerelles.

**1. Les activités.** Les activités sont représentées par un rectangle aux coins arrondis. Elles correspondent à une action pouvant être réalisée par un humain ou par une machine. Une activité peut être atomique ou composée. Les principaux types d'activités sont les suivants :

— **Tâche** : une tâche est représentée par un rectangle à coins arrondis tracé avec une seule ligne mince. Elle correspond à une activité atomique ayant un début et une fin. Une tâche ne peut débuter que si la tâche précédente est terminée. Il existe plusieurs types de tâches :
  — **Tâche utilisateur** : elle précise que la tâche est effectuée par un exécutant humain avec l'aide d'une application informatique.
  — **Tâche manuelle** : elle précise que la tâche doit être effectuée sans l'aide d'un moteur d'exécution de processus métier ni d'une application. Elle est donc réalisée exclusivement par un acteur humain.

— **Tâche service** : elle utilise un service, qui peut être un service Web ou une application automatisée. Il s'agit d'une tâche automatisée sans intervention humaine.

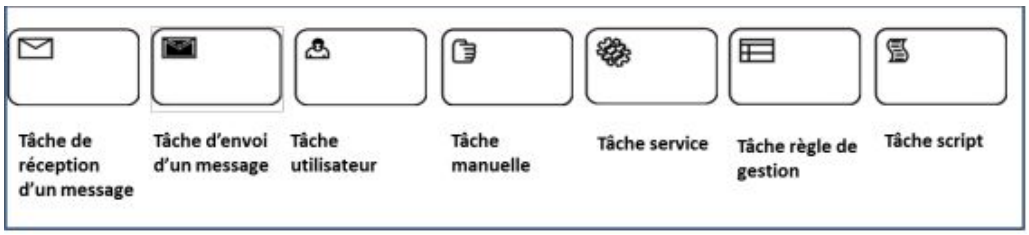



**FIGURE 5 –** *Les types des tâches dans BPMN 2.0.*

**2. Les événements.** Les événements sont représentés par des cercles. Ils correspondent à une action qui survient durant le processus. Ils ont généralement une cause et une conséquence. Il existe trois catégories d'événements : les événements de début, les événements intermédiaires et les événements de fin.




**FIGURE 6 –** *Événements dans BPMN 2.0.*

— **Les événements de début** : ils sont représentés par un cercle avec une seule ligne mince. Ils déclenchent une nouvelle instance de processus. Un processus peut avoir plus d'un événement de début, mais un sous-processus d'événement n'a qu'un seul événement de début.
— **Les événements intermédiaires** : ils sont représentés par un cercle avec une double ligne mince. Ils indiquent une action qui se produit ou peut se produire au cours du processus, entre le début et la fin. Ils peuvent être utilisés pour attraper un déclencheur d'événement et peuvent être placés dans le flux ou à la limite d'une activité.
— **Les événements de fin** : ils sont représentés par un cercle avec une ligne épaisse. Ils marquent la fin de l'instance de processus et ne possèdent pas de flux de séquence sortant.

**3. Les passerelles.** Les passerelles, ou branchements *(Gateways)*, sont représentées par un losange. Elles servent à représenter les conditions de routage entre les flux d'entrée et les flux de sortie. Les signes internes indiquent le type de contrôle du comportement du branchement.




**FIGURE 7 –** *Les branchements de BPMN 2.0.*

Les différents branchements possibles sont :

— **Exclusive (XOR)** : elle est utilisée pour créer des chemins alternatifs dans un flux de processus. Lors de la scission, elle dirige le flux vers une branche sortante. Lors de la fusion, elle attend qu'une branche entrante se termine avant de déclencher le flux de sortie.

— **Inclusive (OR)** : elle peut être utilisée pour créer des chemins alternatifs parallèles. Contrairement à la passerelle exclusive, toutes les expressions de condition sont évaluées. Lors de la scission, elle active une ou plusieurs branches. Lors de la fusion, elle attend que toutes les branches entrantes se terminent avant de fusionner.

— **Parallèle (AND)** : elle est utilisée pour synchroniser et créer des flux parallèles. Lors de la scission, elle active toutes les branches sortantes simultanément. Lors de la fusion, elle attend que toutes les branches entrantes se terminent.

### B. Les objets de connexion

Les objets de connexion sont les éléments graphiques qui permettent de relier les objets de flux les uns aux autres pour représenter le cheminement du processus. Il existe trois types d'objets de connexion.

— **Les flux de séquence** : ils sont représentés par une flèche pleine et une ligne solide. Ils indiquent l'ordre d'exécution des actions, des événements ou des activités.




**FIGURE 8 –** *Les flux de séquence de BPMN 2.0.*

— **Les flux normaux** : le flux normal fait référence aux chemins du flux de séquence qui ne commencent pas à partir d'un événement intermédiaire attaché à la limite d'une activité.

### C. Les partitions

Les partitions sont les éléments graphiques permettant de structurer et de regrouper les différents éléments qui composent le diagramme BPMN. Les deux types d'objets sont les participants et les couloirs.

— **Les participants (Pool)** : un pool est la représentation graphique d'un participant à une collaboration. Un participant peut être une entité partenaire spécifique, par exemple une entreprise, ou un rôle partenaire plus général, comme un acheteur, un vendeur ou un fabricant. Un pool peut avoir des détails internes sous forme d'un processus exécuté ou ne présenter aucun détail interne ; dans ce cas, il est considéré comme une « boîte noire ».

— **Les couloirs (Lane)** : un couloir est un rectangle aux coins carrés pouvant être vertical ou horizontal. Il constitue une partition en sous-ensemble dans le pool. Les couloirs sont utilisés pour organiser et classer les activités. Pour illustrer les échanges entre deux couloirs, on utilise des flux de messages.




**FIGURE 9 –** *La représentation d'un pool avec deux lanes en BPMN 2.0.*

### D. Les données

Les données sont utilisées pour expliquer quelles informations sont nécessaires dans le diagramme.




**FIGURE 10 –** *Les données de BPMN 2.0.*

— **Données (Data)** : objets utilisés pour représenter les données nécessaires à l'exécution d'une activité, ainsi que celles produites. Les objets de données peuvent représenter un objet singulier ou une collection d'objets.

— **Entrée de données (Data Input)** : elle correspond à une entrée externe pour l'ensemble du processus, pouvant être lue par une activité.

— **Sortie de données (Data Output)** : elle correspond au résultat de l'ensemble du processus.

### E. Les artéfacts

Les artéfacts sont des objets supplémentaires qui apportent des détails complémentaires afin d'assurer une meilleure compréhension du processus. Il existe deux types d'artéfacts : le groupe et l'annotation textuelle.




**FIGURE 11 –** *Les artéfacts.*

— **Groupe** : un groupe est un rectangle à coins arrondis tracé avec une ligne pointillée continue. Il permet de regrouper des tâches d'une même catégorie afin de mieux les repérer visuellement sur le diagramme.
— **Annotation textuelle** : elle permet d'ajouter des commentaires pour faciliter la lecture du diagramme BPMN.

#### I.3.1.4. Types de diagrammes BPMN

BPMN distingue principalement les diagrammes de processus, les diagrammes de collaboration et les diagrammes de chorégraphie [3].

— Le **diagramme de processus** décrit le déroulement interne d'un processus dans un participant donné. Il est adapté lorsque l'on souhaite comprendre l'enchaînement des activités d'un service ou d'un système.

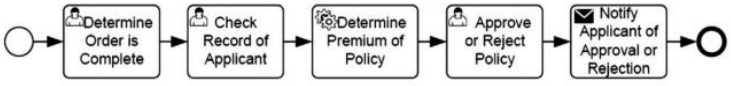



**FIGURE 12 –** *Exemple de diagramme de processus*

— Le **diagramme de collaboration** met en relation plusieurs participants et montre les messages qu'ils échangent. Les Pools sont définis comme étant les participants de cette collaboration. Les messages échangés entre les participants sont présentés à l'aide des flux de messages qui connecte deux pools (ou les objets au sein des pools).

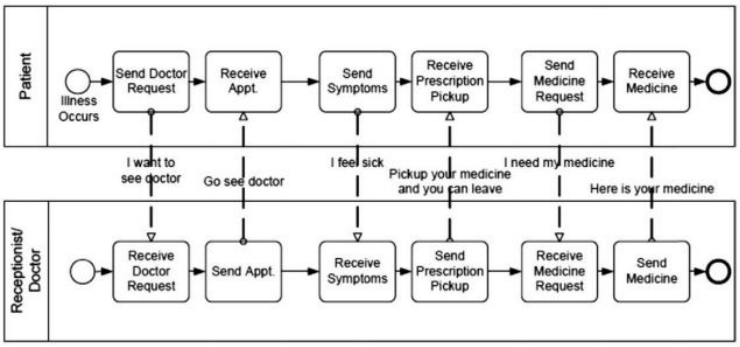



**FIGURE 13 –** *Exemple de diagramme de collaboration*

— Le **diagramme de chorégraphie** se concentre davantage sur les interactions entre participants que sur l'organisation interne de chacun. Dans ce type de modélisation, la focalisation n'est pas sur l'orchestration (c'est-à-dire sur la manière dont le travail est accompli selon le point de vue des participants), mais sur les échanges de Messages entre les Participants.

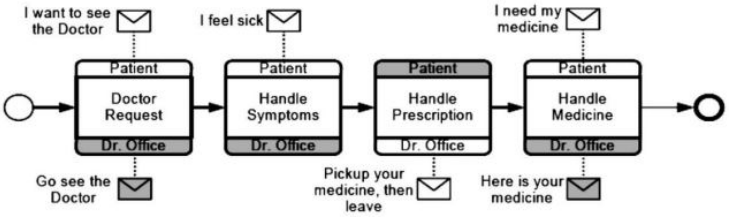



**FIGURE 14 –** *Exemple de diagramme de chorégraphie*

## I.4. Ingénierie dirigée par les modèles

### I.4.1. Principe général de l'ingénierie dirigée par les modèles

L'ingénierie dirigée par les modèles (IDM) désigne une démarche de développement dans laquelle le modèle devient un artefact de travail central. Le modèle n'est donc pas seulement un dessin destiné à expliquer une idée : il sert à décrire un système à un niveau d'abstraction choisi, à vérifier certains choix, à organiser les connaissances du domaine et, lorsque l'outillage le permet, à produire d'autres modèles ou des artefacts techniques. Cette approche répond principalement à deux difficultés du génie logiciel : la croissance de la complexité des systèmes et l'évolution rapide des technologies. En plaçant les modèles au centre du développement, l'IDM cherche à mieux séparer les préoccupations, à réduire la dépendance immédiate aux détails techniques et à rendre certaines tâches répétitives plus maîtrisables

[5, 16]. L'objectif n'est pas de supprimer l'intervention humaine, mais de déplacer une partie de l'effort vers des modèles explicites, contrôlables et transformables.

### I.4.2. Notion de base en Ingénierie Dirigée par les Modèles

Un système est l'entité étudiée. Il peut être composé d'éléments reliés entre eux, posséder une organisation interne et interagir avec son environnement. Dans l'IDM, le système n'est généralement pas manipulé directement dans toute sa complexité ; il est représenté par un ou plusieurs modèles.

Un modèle est une abstraction d'un système, construite dans un objectif donné. Il retient les informations utiles pour répondre à certaines questions, tout en laissant de côté les détails qui ne sont pas nécessaires au point de vue choisi. Cette idée est importante : un modèle n'a pas vocation à tout contenir. Sa valeur dépend de sa pertinence par rapport au besoin d'analyse, de conception ou de vérification. Jézéquel, Combemale et Vojtisek définissent ainsi le modèle comme un ensemble de faits caractérisant un aspect d'un système selon un objectif précis [5].

Un méta-modèle définit le langage dans lequel les modèles sont exprimés. Il précise les concepts autorisés, leurs relations et les contraintes qui structurent les modèles valides. Le méta-modèle joue donc un rôle de cadre : il rend les modèles manipulables par des outils, car il décrit leur structure attendue. Au-dessus, le méta-méta-modèle fournit le langage permettant de définir les méta-modèles. L'Object Management Group (OMG) organise cette logique selon quatre niveaux : M0 pour le système réel, M1 pour les modèles, M2 pour les méta-modèles et M3 pour le méta-méta-modèle, généralement associé à MOF, Meta-Object Facility [5, 17].

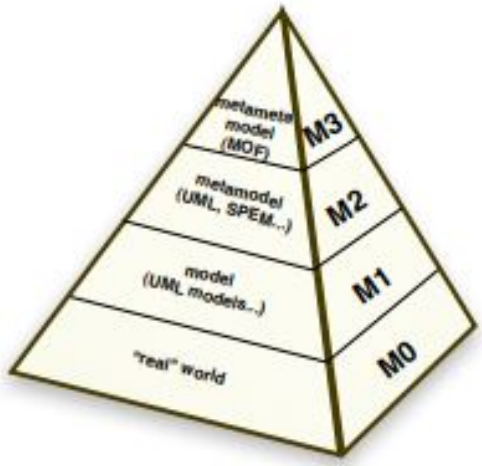



**FIGURE 15 –** *Pyramide de modélisation de l'OMG.[5].*

### I.4.3. Transformation de modèles

La transformation de modèle consiste à produire un modèle cible à partir d'un modèle source, selon des règles définies. Ces règles décrivent les correspondances entre les éléments du modèle d'origine et ceux du modèle obtenu. Une transformation peut être endogène lorsque les modèles source et cible sont conformes au même méta-modèle ; elle est exogène lorsqu'ils relèvent de méta-modèles différents. Elle peut aussi être horizontale lorsqu'elle reste au même niveau d'abstraction, ou verticale lorsqu'elle fait évoluer le niveau d'abstraction, par raffinement ou par abstraction [5, 18].

Les transformations occupent une place centrale dans l'IDM parce qu'elles donnent un caractère productif aux modèles. Elles peuvent servir à raffiner une spécification, restructurer une représentation, migrer d'une technologie à une autre, générer du texte ou produire du code. Les deux grandes familles généralement retenues sont les transformations modèle-vers-modèle (M2M) et modèle-vers-texte (M2T). Les standards et langages de transformation, comme QVT, ATL ou les approches par templates, cherchent à formaliser ces passages afin qu'ils soient reproductibles et moins dépendants d'une manipulation manuelle [5, 19].

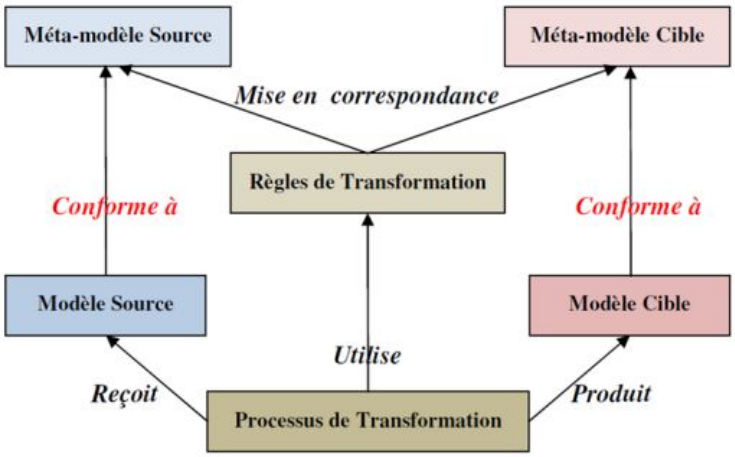



**FIGURE 16 –** *Schéma de base d'une transformation de modèle [16]*

### I.4.4. Model-Driven Architecture

La Model-Driven Architecture (MDA) est une initiative de l'OMG qui applique les principes de l'IDM à l'architecture logicielle. Elle vise à décrire une application en distinguant les modèles indépendants de la plateforme et les modèles liés à une plateforme particulière.

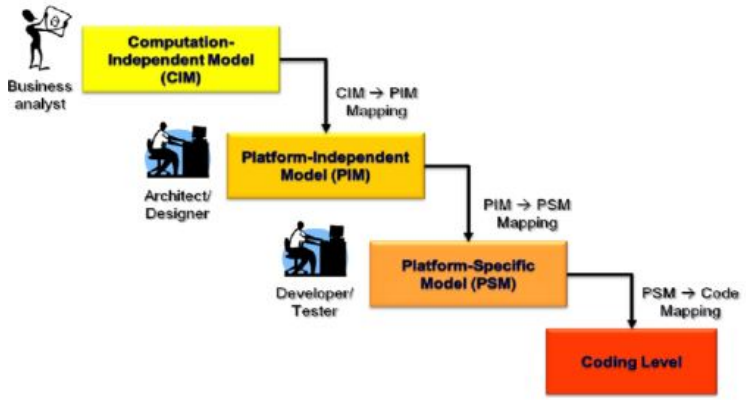



**FIGURE 17 –** *Illustration des trois niveaux d'abstraction du MDA [16]*

— **CIM** ***(Computation Independent Model)*** : il décrit les besoins métier et les objectifs sans se préoccuper des aspects techniques. C'est le niveau où l'on formalise ce que le système doit faire, indépendamment de la façon dont il sera implémenté.
— **PIM** ***(Platform Independent Model)*** : ce modèle représente l'architecture et les fonctionnalités du système sans faire référence à une technologie ou une plateforme donnée. Il constitue un pont entre les exigences métier et la solution technique.
— **PSM** ***(Platform Specific Model)*** : il traduit le PIM en une implémentation concrète, en intégrant les contraintes et les choix technologiques spécifiques comme le framework, le langage ou l'environnement d'exécution.

L'intérêt de la MDA est de préserver les décisions de conception à un niveau moins fragile que le code. Lorsque les technologies changent, les modèles indépendants peuvent être conservés plus longtemps, tandis que les transformations et les modèles spécifiques sont adaptés aux nouvelles plateformes. La MDA ne se limite donc pas à une génération automatique de code ; elle propose une organisation du développement où la séparation entre abstraction, plateforme et implémentation devient explicite.

## I.5. Applications web métier

### I.5.1. Généralités sur le développement web

#### I.5.1.1. Historique du développement web

Le développement web est né avec un objectif essentiellement documentaire : publier des informations et permettre leur consultation à distance. Dans cette pre-

mière logique, le navigateur demandait une page, le serveur retournait un document HTML, puis le navigateur l'affichait. Les fondements du Web restent liés à cette organisation : identification des ressources, liens hypertextes, représentations et échanges entre clients et serveurs [20].

L'évolution des usages a progressivement transformé le Web en plateforme applicative. Les pages statiques ont été complétées par des formulaires, des espaces authentifiés, des tableaux de bord, des services de paiement, des plateformes universitaires et des applications administratives. Le navigateur n'est alors plus seulement un lecteur de documents ; il devient l'interface d'un système logiciel distant. Cette évolution a donné plus d'importance aux langages côté client, aux serveurs d'application, aux bases de données, aux API, à la sécurité et à la maintenabilité.

HTTP demeure le support principal de ces échanges. Le RFC 9110 le définit comme un protocole applicatif fondé sur des requêtes, des réponses et des méthodes comme `GET`, `POST`, `PUT`, `PATCH` ou `DELETE` [21]. Cette base explique pourquoi les applications web modernes sont généralement construites autour de routes, de ressources, d'opérations métier et de formats de représentation comme HTML ou JSON.

#### I.5.1.2. Définition d'application web et concepts du web

##### Définition

Une application web peut être définie comme un logiciel accessible à partir d'un navigateur ou d'un client reposant sur les technologies du Web. Contrairement à un site essentiellement informatif, elle prend en charge des interactions avec l'utilisateur, traite des données, applique des règles métier et produit des résultats exploitables. Ainsi, une plateforme de demande d'attestation ne se limite pas à afficher des pages : elle permet de soumettre une demande, de suivre son état, de contrôler les droits d'accès et de générer un document. Elle relève donc pleinement d'une application web métier.

##### Concepts du web

La conception d'une application web repose sur des choix architecturaux qui influencent directement sa maintenabilité, son évolutivité et la séparation des responsabilités. Le modèle client-serveur constitue la base de ce fonctionnement : le client, généralement un navigateur, transmet les actions de l'utilisateur, tandis que le serveur traite les requêtes, accède aux données et retourne les réponses appropriées [22]. Cette séparation est particulièrement importante dans les applications

métier, où plusieurs utilisateurs interagissent avec les mêmes ressources selon des droits et des règles différents.

Pour améliorer l'organisation interne du système, les applications web modernes s'appuient souvent sur une architecture en couches. La couche de présentation gère les interfaces et les formulaires, la couche métier applique les règles du domaine, et la couche d'accès aux données assure la communication avec la base. Cette structuration limite le couplage entre les composants et facilite les évolutions du système. Le patron Modèle-Vue-Contrôleur s'inscrit dans cette logique en distinguant les données et règles du domaine, leur présentation à l'utilisateur et le traitement des actions déclenchées par celui-ci [23].

Dans les architectures récentes, les applications monopages *(Single Page Applications)* renforcent la séparation entre le client et le serveur en confiant une partie importante de l'interface au navigateur. Elles offrent une expérience plus fluide, mais exigent une maîtrise accrue de la complexité du frontend, notamment lorsque l'application doit représenter des tâches, des états de dossier ou des actions liées à un processus métier [24]. Les API web jouent alors un rôle central en exposant les fonctionnalités du système sous forme de ressources accessibles, comme `/demandes`, `/documents` ou `/taches`. Le style REST, formalisé par Fielding, a fortement contribué à cette organisation des échanges autour des ressources, des représentations et d'interactions sans état [25].

Ainsi, le développement d'une application web ne consiste pas seulement à produire des interfaces. Il implique de définir une architecture capable d'articuler présentation, logique métier, persistance des données et échanges entre composants. Cette base conceptuelle est essentielle pour comprendre les applications web orientées processus, dont le comportement dépend étroitement de l'évolution du workflow.

#### I.5.1.3. Frameworks d'application web

Un framework d'application web fournit un cadre de développement, des composants réutilisables et des conventions pour construire une application. Il évite de réécrire à chaque projet les mêmes mécanismes : routage, contrôleurs, accès aux données, gestion des sessions, validation, rendu des vues, authentification ou traitement des erreurs. Son intérêt n'est pas seulement de gagner du temps ; il permet aussi d'organiser le code de manière plus régulière et plus maintenable.

##### Frameworks de développement web back-end

Les frameworks back-end structurent la partie serveur de l'application. Ils organisent les routes, les contrôleurs, les services, les modèles de données et les mé-

canismes de sécurité. Dans une application métier, c'est dans cette partie que sont vérifiées les règles comme l'autorisation d'un utilisateur, la validité d'une demande ou le passage d'un dossier d'un état à un autre. Ils peuvent aussi faciliter l'accès aux bases de données, la journalisation, la gestion des erreurs et l'exposition d'API. Des environnements comme Spring, Django, Laravel ou Express illustrent cette famille d'outils.

#### Frameworks de développement web front-end

Les frameworks front-end structurent l'interface exécutée dans le navigateur. Ils aident à construire des composants réutilisables, gérer les formulaires, afficher les données et réagir aux actions de l'utilisateur. Dans une application orientée processus, cette partie est importante parce qu'elle rend visibles les tâches à traiter, les états du dossier, les messages d'erreur et les actions autorisées. Leur rôle est donc aussi ergonomique : ils aident à présenter le workflow de façon compréhensible pour chaque acteur. Des outils comme Angular, React ou Vue.js sont représentatifs de cette logique côté client.

### I.5.1.4. Sécurité dans les applications web

Les politiques de contrôle d'accès permettent de définir les règles qui encadrent l'accès aux informations au sein d'une entreprise. Dans les applications web, elles contribuent à sécuriser la manipulation des données sensibles, notamment les données des utilisateurs. Parmi ces politiques, le modèle RBAC, ou contrôle d'accès basé sur les rôles, est largement utilisé, car il permet d'organiser les droits d'accès en fonction des responsabilités attribuées aux utilisateurs.

Le principe du RBAC consiste à associer des permissions à des rôles, puis à affecter les utilisateurs aux rôles correspondant à leurs besoins. Cette approche simplifie la gestion de la sécurité, car les administrateurs peuvent créer des rôles, leur attribuer des autorisations et contrôler ainsi l'accès aux ressources. Elle favorise également des principes essentiels comme le moindre privilège, la séparation des tâches et la protection des informations sensibles.

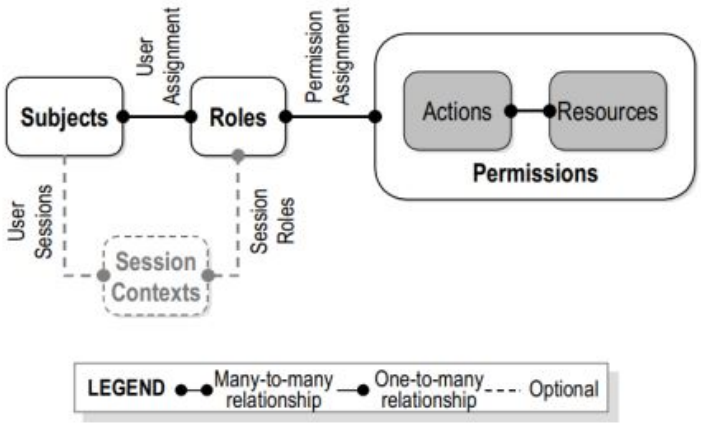



**FIGURE 18 –** *Coeur du modèle RBAC [26]*

## I.5.2. Applications web métier

### I.5.2.1. Définitions et principe

Une application web orientée processus, ou *process-aware web application*, est une application dont le comportement est organisé autour d'un processus métier explicite. Les écrans, les actions disponibles, les rôles des utilisateurs et les changements d'état suivent l'avancement d'un workflow. Cette idée rejoint la définition des systèmes d'information orientés processus, dans lesquels les activités, les acteurs et les règles d'enchaînement occupent une place centrale [1, 2].

Dans une application classique, le développeur peut coder directement les conditions de passage d'un état à un autre. Dans une application orientée processus, ces passages sont plutôt décrits par un modèle, souvent en BPMN. Le modèle précise les tâches à réaliser, les décisions possibles, les événements attendus et les responsabilités des acteurs. L'application web fournit ensuite les interfaces, les services et les données nécessaires à l'exécution concrète de ce processus.

### I.5.2.2. Moteur de processus

Un moteur de processus est un composant logiciel chargé d'exécuter un modèle de processus. Lorsqu'un processus BPMN exécutable est déployé, il peut créer des instances, enregistrer leurs variables, distribuer des tâches humaines, déclencher des services automatiques et suivre l'historique d'exécution [27]. Il joue donc un rôle d'orchestration : il indique ce qui doit être fait, par qui, à quel moment et selon quelles conditions.

Dans une application web orientée processus, le moteur ne remplace pas le frontend, le backend ni la base de données. Il coordonne le déroulement du travail. Le frontend permet aux utilisateurs d'interagir avec les tâches, le backend applique

les règles propres à l'application et dialogue avec le moteur, tandis que la base de données conserve les informations métier. Cette répartition évite de confondre l'exécution du workflow avec la construction complète de l'application.

## I.6. Conclusion

Dans ce chapitre, nous avons présenté les concepts nécessaires à la suite du mémoire : processus métier, BPM, BPMN, développement web et applications web métier. Nous avons montré qu'un modèle BPMN peut guider l'organisation du travail, mais qu'une application web doit aussi gérer l'interface, les données, la sécurité et l'exécution technique. Le chapitre suivant s'appuie sur ces bases pour analyser les travaux existants.

# II

CHAPITRE

# ÉTAT DE L'ART SUR LA GÉNÉRATION D'APPLICATIONS WEB PILOTÉES PAR DES MODÈLES

SOMMAIRE


II.1 - Introduction . . . . . 26
II.2 - Démarche d'analyse des travaux existants . . . . . 26
II.3 - Revue de la littérature . . . . . 27
II.4 - Discussion et positionnement de la recherche . . . . . 36
II.5 - Conclusion . . . . . 38


## II.1. Introduction

La génération d'applications web à partir de modèles de processus se situe au croisement du BPM, de BPMN et du développement web. BPMN permet de représenter le déroulement du travail, mais il ne suffit pas à décrire les interfaces, les données, les règles d'accès et l'architecture d'une application complète [3].

Ce chapitre analyse donc les travaux qui rapprochent les modèles de processus des artefacts logiciels, afin d'en dégager les apports, les limites et le positionnement de notre recherche.

## II.2. Démarche d'analyse des travaux existants

La revue n'a pas cherché à rassembler tous les travaux sur BPMN, mais à analyser ceux qui répondent à une question centrale : comment un modèle de processus

peut-il être transformé en application web ou en artefacts logiciels utilisables ? Le corpus retenu privilégie donc les travaux portant sur les moteurs de workflow, les transformations BPMN vers d'autres modèles, la génération de code, les applications web générées, les plateformes orientées processus et les approches low-code. La démarche s'inspire des recommandations de Kitchenham et Charters ainsi que de l'esprit de PRISMA, sans constituer une revue systématique exhaustive [28, 29].

L'analyse des travaux s'est faite en trois étapes : comprendre l'objectif de chaque approche, identifier les artefacts réellement produits et examiner les limites récurrentes. Ces limites concernent notamment l'insuffisance d'informations dans les modèles BPMN, la couverture partielle des applications, la dépendance à certaines plateformes, les difficultés de régénération et la cohabitation entre code généré et code manuel.

**TABLE II –** *Grille concrète de lecture des travaux étudiés*

| Point observé | Question posée pendant la lecture |
|---|---|
| Point de départ | Le travail part-il d'un modèle BPMN seul, d'un processus enrichi, d'un modèle de données ou d'un autre artefact ? |
| Résultat obtenu | Produit-il une interface, une API, une application exécutable, un prototype ou seulement une méthode de transformation ? |
| Place de la plateforme | Le résultat dépend-il fortement d'un moteur, d'un framework ou d'un environnement low-code ? |
| Preuve apportée | Les auteurs valident-ils leur proposition par un cas d'étude, une expérimentation, une comparaison ou une revue ? |
| Limite retenue | Que reste-t-il à faire pour obtenir une application complète, maintenable et adaptable ? |

La revue qui suit présente donc les travaux par familles, puis met en évidence les limites qui se dégagent de leur comparaison.

## II.3. Revue de la littérature

### II.3.1. Systèmes d'information orientés processus et plateformes d'exécution

Les systèmes d'information orientés processus placent le déroulement du travail au centre de l'organisation du logiciel. Ils s'inscrivent dans le domaine plus large de la gestion des processus métier, qui couvre la découverte, la modélisation, l'exécution, le suivi et l'amélioration des processus [30, 31]. Dans cette perspec-

tive, le modèle ne sert plus uniquement à documenter une organisation : il peut devenir une définition exécutable utilisée par un moteur de processus.

#### II.3.1.1. Moteurs BPMS et plateformes d'exécution

Les systèmes de gestion des processus métier, ou *Business Process Management Systems* (BPMS), tels que Camunda, Flowable, jBPM ou Activiti interprètent des définitions de processus, créent des instances, distribuent des tâches et conservent un historique d'exécution [27, 32–34]. Leur principal apport réside dans la séparation entre la logique d'orchestration et les services appelés au cours du processus. Cette séparation rend le déroulement visible et facilite son suivi. Elle impose néanmoins aux applications clientes de manipuler les notions propres au moteur, notamment les instances, les variables et les tâches. La plateforme exécute ainsi le processus sans nécessairement produire l'ensemble de l'application qui l'entoure.

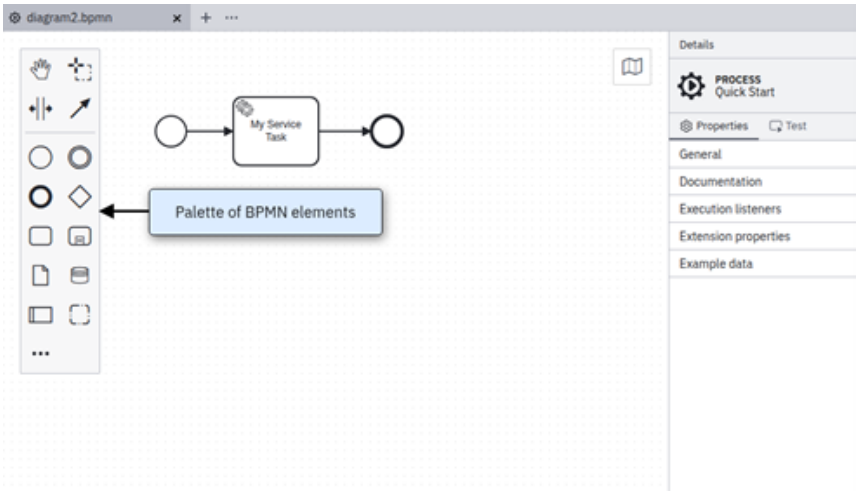



**FIGURE 19 –** *Modélisation d'un processus BPMN exécutable dans Camunda Modeler, d'après la documentation Camunda [27].*

La figure 19 montre le point d'entrée d'un moteur comme Camunda : un processus BPMN enrichi de propriétés exécutables. Une fois déployé, le moteur crée les instances, affecte les tâches et suit l'exécution, mais il ne génère pas à lui seul toute l'application web qui l'exploite.

#### II.3.1.2. Architecture web pilotée par BPMN

Schäffer et al. proposent une architecture de référence et une méthode de développement agile pour une plateforme web pilotée par BPMN [35]. Leur travail montre comment associer une application web, un moteur de processus et des services afin de conserver le processus comme fil conducteur de l'implémentation.

L'approche apporte une organisation cohérente aux systèmes dont le comportement dépend fortement d'un workflow. Elle reste toutefois centrée sur l'exécution et l'intégration du processus : les autres dimensions de l'application doivent toujours être conçues et développées autour de cette architecture.

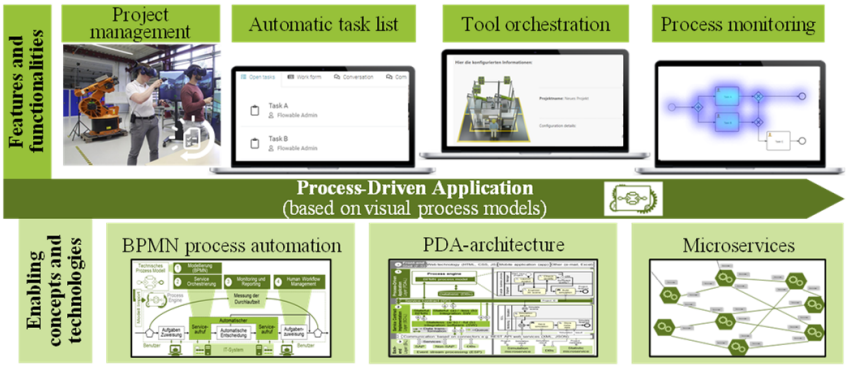



**FIGURE 20 –** *Fonctionnalités et technologies d'une application web pilotée par processus [35].*

La figure 20 montre qu'une application pilotée par processus combine moteur BPMN, tâches, orchestration, supervision et services. Le moteur de workflow y apparaît central, mais il doit être entouré de composants applicatifs et d'intégration.

### II.3.1.3. WebWorkFlow

WebWorkFlow adopte une direction différente. Hemel et al. considèrent qu'un langage de workflow isolé n'est pas suffisant pour générer une application web complète. Ils étendent donc WebDSL avec des abstractions de workflow, puis traduisent ces abstractions vers les constructions du langage sous-jacent [36]. Cette intégration permet de générer des applications personnalisées tout en couvrant les principaux patrons de contrôle de workflow.

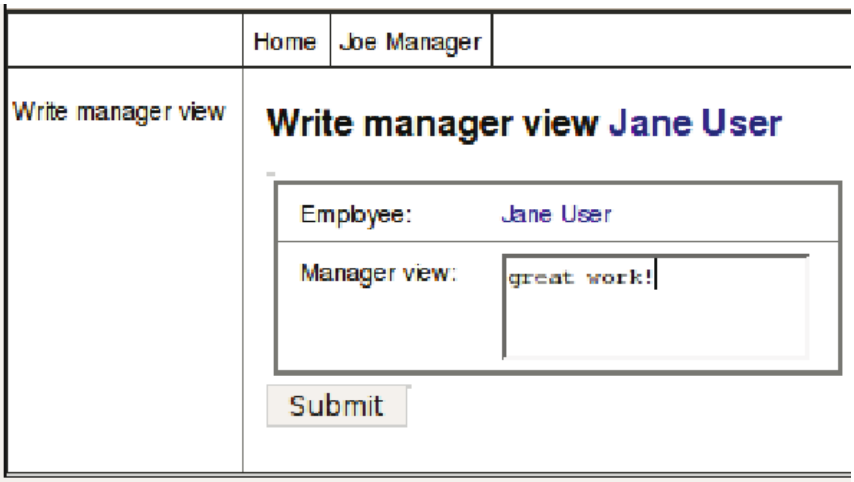



**FIGURE 21 –** *Interface générée dans WebWorkFlow pour une tâche de workflow, extraite de [36].*

La figure 21 illustre le résultat attendu : une tâche de workflow devient une interface exploitable. L'apport est concret, mais la portée de la génération reste liée au langage et aux abstractions de WebDSL.

#### II.3.1.4. Plateforme AKIP

La plateforme ouverte AKIP, nom donné par ses auteurs, cherche à produire des applications web orientées processus à partir de BPMN exécutable et de spécifications complémentaires [37]. Son architecture associe un générateur, un frontend, un backend, un moteur de processus, des connecteurs et plusieurs fonctions transversales. Les auteurs répondent notamment aux limites observées dans des plateformes existantes : dépendance envers un fournisseur, interfaces peu flexibles, difficultés d'intégration et technologies devenues anciennes.

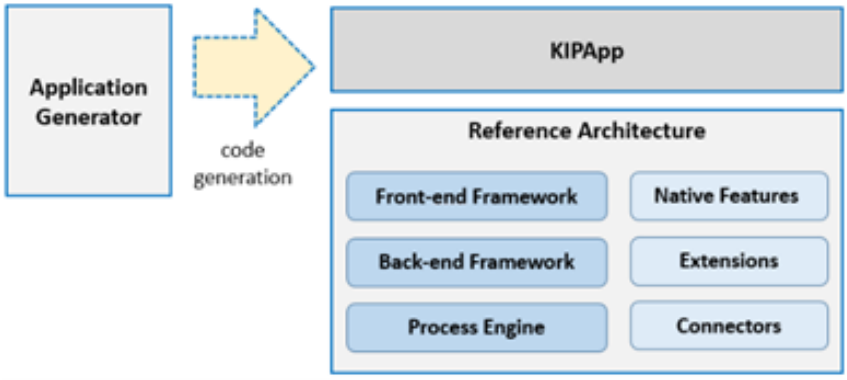



**FIGURE 22 –** *Architecture et principe de génération d'une application dans AKIP, adaptés de [37].*

La figure 22 montre une génération organisée autour d'une architecture de référence. AKIP produit une base applicative moderne, mais son évaluation reste limitée à une organisation et certaines constructions BPMN ne sont pas encore couvertes.

Ces travaux montrent que l'exécution d'un processus est aujourd'hui bien maîtrisée. Ils montrent également que le moteur ne résout qu'une partie du problème applicatif. Plus une proposition vise une application complète, plus elle doit associer au workflow des informations sur le domaine, les interfaces et l'intégration technique.

### II.3.2. Dérivation d'artefacts applicatifs à partir de BPMN

Une deuxième famille de recherches utilise BPMN comme point de départ pour dériver des artefacts web précis. Ces travaux ne cherchent pas toujours à générer l'ensemble d'un système. Ils étudient plutôt la manière dont les éléments du pro-

cessus peuvent fournir des informations utiles à la conception d'interfaces, d'API ou de prototypes.

#### II.3.2.1. Transformation vers des modèles d'interaction

Sajji et al. proposent une transformation de BPMN vers l'*Interaction Flow Modeling Language* (IFML) dans le cadre de la *Model Driven Architecture* (MDA) [7]. BPMN représente le niveau indépendant du calcul, tandis qu'IFML décrit les vues, les événements utilisateur et les flux d'interaction au niveau indépendant de la plateforme. Le travail établit des règles de correspondance entre les éléments du processus et ceux de l'interface. Il confirme que la structure du workflow peut guider la construction de la navigation, mais que BPMN doit être complété pour exprimer les détails de l'interaction.

Ces même auteurs poursuivent cette logique en étudiant la transformation d'IFML vers Symfony [38]. L'intérêt de ce prolongement est de montrer que le modèle d'interaction peut servir de point de départ à une génération orientée framework web concret. La transformation s'appuie sur des règles ATL, c'est-à-dire un langage utilisé pour définir des transformations entre modèles, pour passer d'un modèle IFML indépendant de la plateforme vers un modèle Symfony plus proche de l'implémentation.

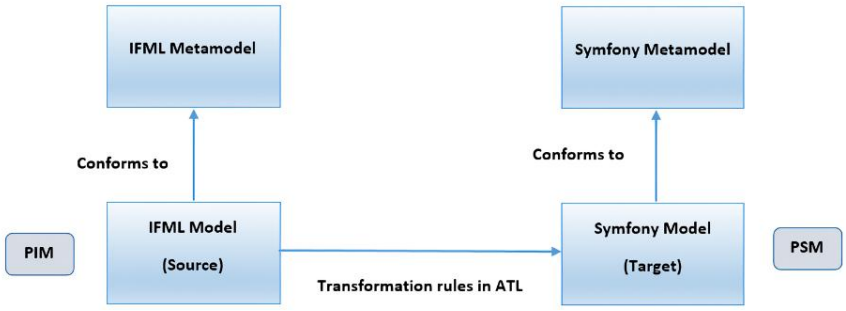



**FIGURE 23 –** *Transformation d'un modèle IFML vers un modèle Symfony, adaptée de [38].*

La figure 23 illustre la logique MDA : IFML sert de modèle indépendant, puis Symfony rapproche la représentation de l'implémentation. Cette continuité réduit l'écart avec le code, mais renforce aussi la dépendance aux conventions du framework cible.

Laaz et al. enrichissent les modèles BPMN avec des ontologies de domaine avant de produire des modèles IFML [39]. Les ontologies permettent de préciser les concepts métier que le processus manipule sans les surcharger directement dans le diagramme. Cette proposition répond à une faiblesse récurrente de BPMN : les activités et les flux sont explicites, mais la sémantique des données demeure souvent implicite. La qualité du résultat dépend toutefois de la disponibilité et de la

qualité de l'ontologie, dont la construction représente elle-même un effort important.

#### II.3.2.2. Génération de prototypes et d'API

Díaz et al. associent les processus BPMN à des modèles structurels afin de générer des prototypes d'interfaces graphiques [6]. Leur travail montre l'intérêt d'un prototype obtenu tôt dans le cycle de développement : il permet de confronter le processus modélisé aux interactions attendues par les utilisateurs. Le terme de prototype marque cependant la limite assumée de la proposition. L'interface obtenue facilite la validation, mais elle ne constitue pas encore une application complète prête à être exploitée.

Pietruck et Steffens s'intéressent à la génération d'API conformes au style architectural *Representational State Transfer* (REST) pour les moteurs de workflow, à partir de modèles de processus et de données [40]. Les API produites exposent des opérations plus proches du domaine métier que les interfaces génériques d'un moteur. Cette abstraction réduit le couplage entre les applications clientes et les concepts techniques de son environnement d'exécution. Elle couvre cependant la couche d'accès au processus et aux données, sans générer les interfaces utilisateur ni l'ensemble des fonctions transversales.

#### II.3.2.3. Vers une génération intégrée des artefacts applicatifs

Hoyos et Restrepo-Calle proposent de générer automatiquement un prototype d'application web orientée processus à partir d'une spécification rédigée dans un langage naturel restreint [41]. Leur chaîne analyse la spécification, extrait les informations utiles, puis produit des classes du domaine, des formulaires, des vues et des contrôleurs.

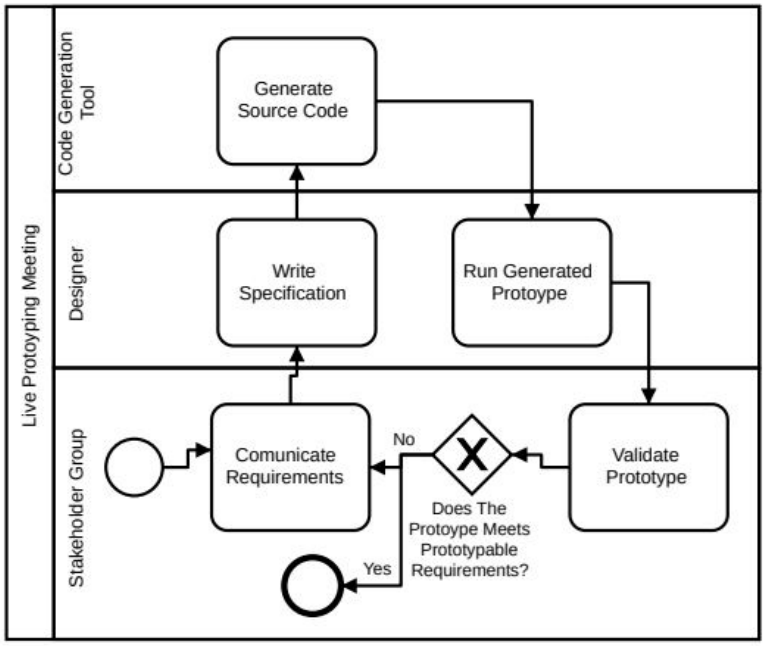



**FIGURE 24 –** *Cycle de prototypage rapide d'applications web proposé par [41].*

```
@app.route('/projectList/<pid>/showSelectedProject',
methods=['GET','POST'])
def projectList_showSelectedProject(pid):
    viewparams={}
    data = get_instance_data(pid)
    formdata = request.form.copy()
    for file in request.files:
        formdata[file] = request.files[file].filename
    request.form = formdata
    project = session.query(Project).filter(
       Project.id==data.get("selected_Project",0)
    ).first()
    viewparams['project']=project
    task_list = session.query(Task).filter(
        Task.project_id==data.get('selected_Project',0)
    ).all()
    viewparams['task_list']=task_list
```

```
<div class="panel-heading">Show Selected Project</div>
<div class="panel-body">
{% for field in project.__table__.columns._data.keys()%}
<div class="form-group">
        <b>{{field}}</b><br /> {{project[field]}}
</div>
{% endfor %}
<div class="col-lg-12">
<div class="table-responsive">
<table class="table">
<tr>
{% for prop in task_list[0].__mapper__.columns.keys() %}
<th>{{prop}}</th>
{% endfor %}
</tr>
{% for ent in task_list %}
<tr>
{% for prop in ent.__mapper__.columns.keys() %}
<td>{{ent[prop]}}</td> {% endfor %}
</tr>
{% endfor %}
</table></div></div></div>
```

**FIGURE 25 –** *Extraits de code généré pour le contrôleur et la vue dans l'approche de [41].*

La figure 25 complète le cycle précédent en montrant deux artefacts produits : une route côté contrôleur et un gabarit de vue. Elle confirme que l'approche génère un prototype exploitable, même si la qualité finale dépend encore des règles de génération.

Uyanik et Sayar développent un environnement web qui réunit conception interactive du processus, gestion des flux et génération automatique de code [42]. Dans leur contexte d'évaluation, les auteurs rapportent une amélioration de 98,68 % du

processus de développement, de 95,84 % pour les conditions de test et de 36,01 % concernant la taille du code. Ces résultats soulignent le potentiel d'une chaîne intégrée pour des applications principalement organisées autour de formulaires et de processus. Ils restent toutefois liés au protocole et au domaine évalués ; ils ne permettent pas d'établir que les mêmes gains seraient obtenus pour des architectures plus variées.

Pris ensemble, ces travaux confirment que BPMN contient des informations exploitables pour produire plusieurs parties d'une application web. Ils montrent aussi qu'une transformation ciblée conserve généralement une couverture ciblée : l'interface, l'API ou le prototype progresse, tandis que d'autres préoccupations restent hors du périmètre.

### II.3.3. Ingénierie dirigée par les modèles et génération de systèmes orientés processus

L'ingénierie dirigée par les modèles considère les modèles comme des artefacts productifs pouvant être validés, transformés et reliés au code. Cette orientation est particulièrement adaptée aux systèmes orientés processus, car elle permet d'associer plusieurs descriptions spécialisées au lieu d'exiger qu'un seul modèle représente toute l'application.

#### II.3.3.1. Composition de langages et génération progressive

Drave et al. proposent une démarche générative pour les systèmes d'information orientés processus [9]. Leur approche combine un langage textuel pour BPMN avec des langages spécifiques décrivant notamment les structures de données et les rôles. Les modèles sont contrôlés, transformés en représentations intermédiaires, puis utilisés pour générer un système extensible. Cette composition de langages vise à couvrir à la fois les activités manuelles et automatisées, là où de nombreuses propositions se concentrent sur une seule de ces dimensions.

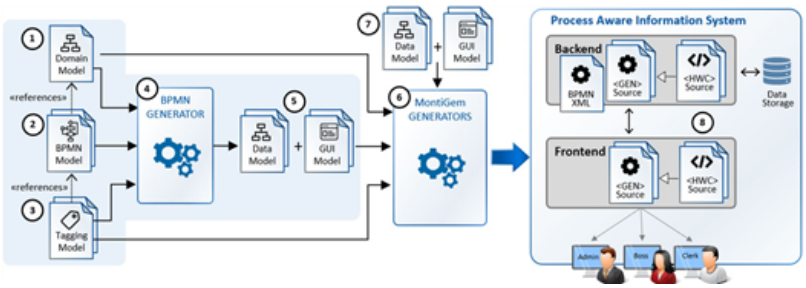



**FIGURE 26 –** *Génération en deux étapes d'un système d'information orienté processus[9].*

La figure 26 montre une génération en deux étapes qui facilite le contrôle des transformations et la réutilisation des informations. Sa limite tient à la couverture incomplète de BPMN et au coût de maintenance des langages spécifiques mobilisés.

#### II.3.3.2. Gabarits et compléments aux modèles BPMN

La génération fondée sur des templates, c'est-à-dire des gabarits de fichiers complétés automatiquement, reste l'une des techniques les plus utilisées pour passer d'un modèle vers du code. L'étude de cartographie menée par Syriani et al. montre que cette technique est répandue et qu'elle facilite l'adaptation aux technologies cibles [43]. Dans le cadre d'une génération d'applications web métier, les gabarits ne sont toutefois utiles que s'ils traduisent correctement les informations du processus en artefacts concrets : routes, contrôleurs, formulaires, services, règles métier et accès aux données. Leur maintenance devient alors un enjeu, car ils doivent évoluer avec les cadres logiciels, les conventions de développement et la structure des modèles d'entrée.

Plusieurs travaux montrent que le modèle de processus doit souvent être complété pour produire une application exploitable. Fedeli et al. introduisent des concepts propres aux environnements enrichis par l'Internet des objets [44], tandis que Salado-Cid et al. insistent sur la représentation explicite des données, des opérations et des dépendances dans les workflows intensifs en données [45]. Ces contributions ne visent pas directement les applications web métier classiques, mais elles soulignent une limite importante de BPMN : le flux de contrôle ne suffit pas toujours à décrire les données manipulées, les interactions utilisateur et les contraintes métier nécessaires à la génération.

Les travaux sur les familles de processus BPMN complètent ce constat. Delgado et al. montrent l'intérêt de représenter des variantes proches d'un même processus lorsque les organisations, les règles ou les contextes d'exécution diffèrent [46].

### II.3.4. Maintenabilité du code généré et mise en perspective avec le low-code

La plupart des travaux de génération accordent une attention importante au premier passage du modèle vers le code. Or, une application continue d'évoluer après sa génération initiale. Les développeurs ajoutent des règles propres au domaine, corrigent des comportements et adaptent les interfaces. En parallèle, les processus et les technologies cibles changent. La coexistence du code généré et du code écrit manuellement devient alors une question centrale.

#### II.3.4.1. Gestion conjointe du code généré et du code manuel

Greifenberg et al. comparent plusieurs mécanismes d'intégration du code manuscrit et du code généré, notamment la séparation des fichiers, l'héritage, la délégation et les zones protégées [47]. Leur analyse montre qu'aucun mécanisme ne répond parfaitement à toutes les situations. Certains protègent clairement les modifications manuelles, mais imposent une architecture plus complexe. D'autres offrent davantage de souplesse, au prix d'un risque de conflit lors de la régénération. Le choix doit donc être prévu dès la conception du générateur.

Rahad et al. étudient la place du développeur dans la boucle de l'ingénierie dirigée par les modèles [48]. Ils montrent que l'automatisation ne supprime pas l'intervention humaine : celle-ci se déplace vers la vérification des modèles, l'extension du code et la résolution des écarts entre le résultat généré et le besoin réel. SeamlessMDD poursuit une préoccupation similaire en cherchant à intégrer plus naturellement le code généré aux pratiques habituelles de développement [49]. Ces contributions rappellent que la qualité d'une chaîne générative se mesure aussi après la première génération.

#### II.3.4.2. Mise en perspective avec les approches low-code

Les plateformes low-code cherchent également à réduire l'effort de développement grâce à des modèles visuels, composants et connecteurs [50, 51]. Elles partagent avec l'IDM une recherche d'abstraction et d'automatisation. Toutefois, elles restent souvent dépendantes des plateformes, de leurs métamodèles, de leurs règles de gouvernance et de leurs connecteurs. Elles ne remplacent donc pas l'étude spécifique d'une génération pilotée par BPMN enrichi et contrôlée par des transformations explicites.

## II.4. Discussion et positionnement de la recherche

La littérature étudiée confirme qu'il est possible d'utiliser un modèle de processus pour exécuter un workflow, produire des interfaces, exposer des API ou générer une base applicative. Elle ne fait toutefois pas apparaître une réponse unique couvrant, avec le même niveau de maturité, toutes les dimensions d'une application web. Les propositions se distinguent surtout par le compromis qu'elles établissent entre couverture, automatisation, flexibilité et dépendance technologique.

**TABLE III –** *Comparaison de travaux représentatifs étudiés*

| Travail | Portée et résultat produit | Évaluation présentée | Limite principale |
|---|---|---|---|
| WebWorkFlow [36] | Intégration d'abstractions de workflow à WebDSL pour générer des applications web personnalisées | Couverture des principaux patrons de contrôle de workflow | Dépendance au langage WebDSL et à ses abstractions |
| Sajji et al. [7, 8, 38] | Transformation de BPMN vers IFML, puis d'IFML vers des technologies web comme WebRatio ou Symfony | Cas d'étude d'une application de service après-vente et règles de transformation vers Symfony | Couverture principalement limitée aux interactions et dépendance aux conventions de la plateforme cible |
| Hoyos et Restrepo-Calle [41] | Génération de classes, formulaires, vues et contrôleurs depuis un langage naturel restreint | Étude de cas démontrant la génération rapide d'un prototype | Langage d'entrée contraint et résultat orienté prototypage |
| Pietruck et Steffens [40] | Génération d'API REST orientées métier pour des moteurs de workflow | Démonstration de la transformation de modèles de processus et de données | Interfaces utilisateur et fonctions transversales hors du périmètre |
| Delgado et al. [46] | Gestion dirigée par les modèles des familles de processus BPMN | Représentation de variantes proches d'un même processus | Configuration et contrôle de cohérence plus complexes |
| AKIP [37] | Génération d'une application web fonctionnelle intégrant interface, serveur et moteur de processus | Étude industrielle portant sur trois processus | Personnalisations métier nécessaires et couverture BPMN partielle |
| Uyanik et Sayar [42] | Environnement intégré de conception de processus et de génération de code | Gains importants rapportés pour le développement, les tests et la taille du code | Résultats liés à un contexte expérimental spécifique |
| Travaux low-code [51–55] | Usage de modèles visuels ou d'entreprise pour accélérer la construction d'applications | Revues et études de transformation vers des plateformes low-code | Dépendance aux plateformes, gouvernance et intégration persistantes |

Le tableau III montre que les travaux étudiés apportent des réponses utiles, mais rarement complètes. Certains exécutent le processus, d'autres génèrent une interface, une API ou un prototype. Les approches les plus larges produisent une base applicative, mais elles restent liées à une architecture, à un langage ou à une plateforme particulière.

BPMN apparaît ainsi comme un point d'entrée pertinent, mais insuffisant pour produire seul une application web complète. Les travaux doivent souvent lui associer des modèles d'interaction, des modèles de données, des métadonnées ou des langages propres au domaine. Cette nécessité confirme que la génération ne peut pas reposer uniquement sur le diagramme de processus.

La limite la plus importante concerne l'évolution du résultat généré. Une application web doit pouvoir être modifiée, maintenue et parfois régénérée après un changement du processus. Or cette dimension reste peu stabilisée dans les travaux étudiés. Notre recherche se positionne donc dans cet espace : relier le modèle de processus aux artefacts applicatifs tout en gardant un code compréhensible, modifiable et maintenable.

## II.5. Conclusion

Les travaux étudiés montrent que les modèles de processus peuvent piloter l'exécution d'un workflow et guider la production d'interfaces, d'API ou d'applications web. Ils révèlent cependant plusieurs difficultés persistantes : BPMN doit être complété, la couverture applicative reste fragmentée, les évaluations sont rarement généralisables, la variabilité des processus reste difficile à gouverner et l'évolution du code généré demeure délicate. La cohérence entre le processus, les artefacts logiciels produits et leur maintenance constitue ainsi un problème encore ouvert.

# III

CHAPITRE

# APPROCHE D'INGÉNIERIE PROPOSÉE POUR LA GÉNÉRATION D'APPLICATIONS WEB PILOTÉES PAR BPMN

SOMMAIRE


III.1 - Introduction . . . . . 39
III.2 - Vue d'ensemble de l'approche . . . . . 40
III.3 - Architecture globale du framework proposé . . . . . 41
III.4 - Modèle d'entrée enrichi : spécification spec-v1 . . . . . 43
III.5 - Règles de transformation et génération de l'application web . . . . . 47
III.6 - Préservation du code personnalisé et synchronisation . . . . . 52
III.7 - Conclusion . . . . . 54


## III.1. Introduction

Ce chapitre présente l'approche proposée pour générer une application web métier à partir d'un modèle BPMN complété par une spécification applicative. Cette approche est concrétisée dans ProOWeb, le framework développé dans ce travail.

L'objectif est de montrer comment l'approche conserve la traçabilité entre le processus et le code généré, tout en laissant au développeur la possibilité d'adapter l'application. Le chapitre présente successivement la vue d'ensemble, l'architecture, la spécification d'entrée, les transformations et les mécanismes de préservation du code personnalisé.

## III.2. Vue d'ensemble de l'approche

### III.2.1. Problème traité et principe général

L'approche proposée vise à générer une application web métier à partir d'un processus métier modélisé. ProOWeb doit fournir un socle applicatif que le développeur peut tester, adapter, versionner et maintenir.

Le choix de BPMN comme modèle d'entrée se justifie par sa capacité à représenter les activités, événements, passerelles et flux d'un processus métier . Toutefois, BPMN ne décrit pas toujours les informations nécessaires à la génération d'une application web, telles que les formulaires, les droits d'accès, la persistance ou les règles d'affectation. Pour cette raison, ProOWeb repose sur un double modèle d'entrée : le fichier BPMN décrit le déroulement du processus, tandis que la spécification *spec-v1* fournit les détails applicatifs nécessaires.

Le fonctionnement général suit quatre étapes : analyser le modèle BPMN, lire la spécification *spec-v1*, construire un modèle applicatif intermédiaire, puis générer les artefacts liés à l'interface, à la logique métier, aux données, à la sécurité et au suivi du processus.

### III.2.2. Positionnement dans le cycle MDA

La Model-Driven Architecture fournit un cadre pertinent pour comprendre la progression proposée. ProOWeb ne reprend pas ce cadre comme une classification abstraite ; il l'utilise pour clarifier les responsabilités de chaque niveau de la chaîne de génération.

**TABLE IV –** *Positionnement de ProOWeb dans le cycle MDA*

| Niveau | Contenu dans ProOWeb | Rôle dans la génération |
|---|---|---|
| PIM | BPMN et spécification *spec-v1* | Décrire le workflow, les rôles, les entrées, les sorties, la visibilité et les données sans dépendre directement du code cible. |
| PSM | Contrats intermédiaires, catalogues et registres | Stabiliser les informations dans une forme exploitable par les générateurs d'interface, de logique applicative et de gestion des données. |
| Code | Projet applicatif comprenant interface, logique métier, données, sécurité et configuration | Produire une application web métier complète, fonctionnelle, exécutable, versionnable et personnalisable. |

### III.2.3. Contribution méthodologique de l'approche

La contribution de ce travail se situe à l'intersection de trois préoccupations. La première est la modélisation du processus. Le BPMN reste la référence du workflow et permet de maintenir une continuité avec les pratiques de modélisation métier. La deuxième est la génération d'une application web métier. Le framework ProOWeb ne se limite pas à produire des écrans ou des services isolés ; il organise un projet cible avec ses modules fonctionnels et techniques. La troisième est la maintenance du code généré. La régénération est traitée comme une opération contrôlée, accompagnée par un manifeste, des sauvegardes, des rapports et des mécanismes d'extension.

ProOWeb vise donc à produire un résultat directement exploitable par une équipe de développement et par les acteurs métier : une application web métier fonctionnelle, générée à partir du processus, mais ouverte à l'adaptation.

## III.3. Architecture globale du framework proposé

L'architecture de ProOWeb est organisée autour de quatre parties complémentaires. La première regroupe les modèles d'entrée. La deuxième correspond au noyau de génération du framework. La troisième représente l'application web produite. La quatrième concerne l'exploitation et l'évolution de cette application. Cette lecture par blocs permet de distinguer clairement ce qui est fourni par le concep-

teur, ce qui est traité par ProOWeb, ce qui est généré et ce qui est utilisé ensuite par les acteurs du système.

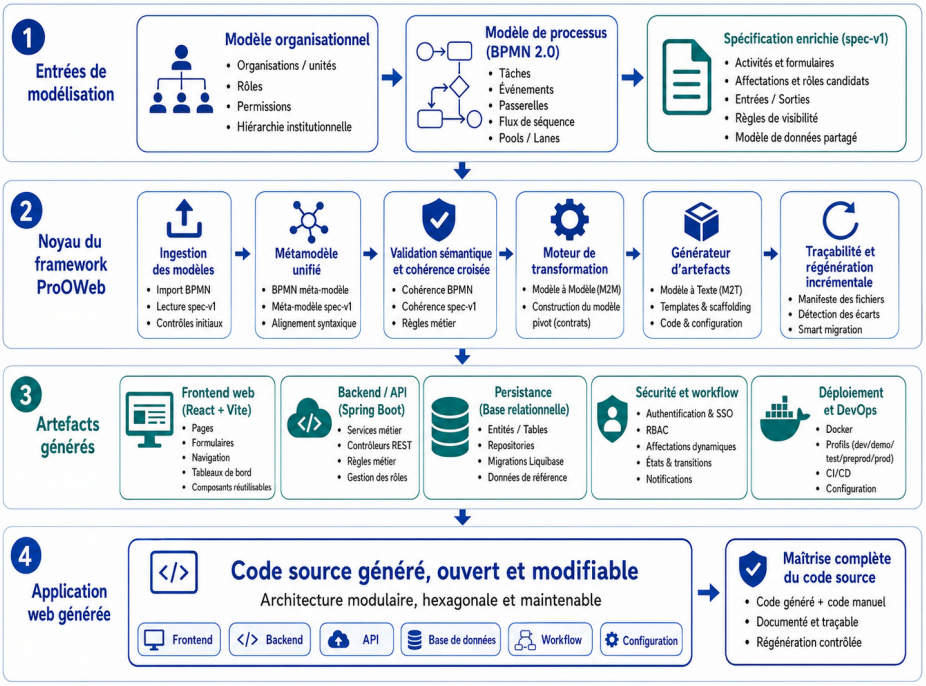



**FIGURE 27 –** *Architecture générale de ProOWeb*

### III.3.1. Modèles d'entrée de ProOWeb

Le premier bloc de l'architecture constitue le point de départ de la génération. Il regroupe le modèle BPMN, la spécification *spec-v1*, les informations organisationnelles et la configuration du projet cible. Le modèle BPMN décrit le déroulement du processus à travers les activités, événements, passerelles et flux, tandis que la spécification *spec-v1* apporte les éléments nécessaires à l'application, notamment les rôles, les données, les règles d'affectation, de stockage et de visibilité. Les informations organisationnelles relient les tâches aux acteurs concernés, alors que la configuration précise les options générales du projet à générer. Ce bloc joue ainsi le rôle de source de vérité : ProOWeb y lit, valide et transforme les informations métier afin de préserver la traçabilité entre le modèle initial et le code produit.

### III.3.2. Noyau de génération du framework

Ce bloc permet de transformer les modèles d'entrée en artefacts générables à travers plusieurs traitements : ingestion du BPMN, enrichissement par la spécification *spec-v1*, validation croisée, construction du modèle pivot et génération des fichiers applicatifs. L'ingestion extrait les activités, événements, passerelles et flux

du processus, puis l'enrichissement associe à chaque activité les rôles, données, règles de visibilité et comportements définis dans la spécification. La validation vérifie ensuite la cohérence entre le BPMN et *spec-v1*, afin d'éviter une application générée fonctionnellement incorrecte. Après cette étape, ProOWeb construit un modèle pivot composé de contrats intermédiaires, tels que le *runtime contract*, le *data contract*, les catalogues d'interface et le registre des processus. Ces contrats structurent les transformations modèle-à-modèle et modèle-à-texte nécessaires à la production du code source.

### III.3.3. Application web générée

Ici, il ne s'agit pas d'un simple assemblage de fragments, mais d'un projet web métier structuré, exécutable et exploitable par les acteurs concernés. L'application générée comprend les vues de traitement, les listes de tâches, les formulaires, les interfaces de suivi et les espaces de supervision. Elle intègre également les services nécessaires au démarrage d'un processus, à l'exécution des activités, à la gestion des transitions, à la persistance des données et au contrôle des droits d'accès. Les données peuvent être rattachées à une instance de processus ou conservées comme informations partagées, selon les règles définies dans *spec-v1*. Ainsi, ProOWeb produit une base applicative complète, reliant interface, logique métier, sécurité, données, exécution du workflow et configuration du projet cible.

### III.3.4. Exploitation et évolution de l'application

Après la génération, l'application est utilisée par les acteurs métier pour exécuter les tâches, par les superviseurs pour suivre les instances, par les administrateurs pour gérer les paramètres et par les développeurs pour adapter le code lorsque des besoins spécifiques apparaissent. Cette dimension évolutive est essentielle, car un processus, une règle d'affectation, un champ ou une interface peuvent être modifiés au cours du temps. ProOWeb prend donc en compte la régénération à travers le suivi des fichiers produits, la détection des modifications manuelles, la création de sauvegardes et la production de rapports de migration. L'architecture forme ainsi une boucle itérative reliant les modèles, le code généré et les adaptations réalisées.

## III.4. Modèle d'entrée enrichi : spécification spec-v1

La spécification *spec-v1* constitue le complément applicatif du modèle BPMN. Elle rend explicites les informations nécessaires à la génération d'une application web métier. Son rôle est donc de décrire ce que BPMN ne porte pas naturelle-

ment : rôles autorisés, supervision, stratégie d'affectation, données d'entrée, sorties, règles de visibilité, stockage et comportement des activités automatiques.

### III.4.1. Structure générale de spec-v1

La structure de *spec-v1* est organisée autour de blocs fonctionnels. Le bloc de démarrage indique les rôles autorisés à lancer une instance et les activités initiales. Le bloc de supervision précise les rôles capables de suivre ou d'administrer le processus. Le bloc des activités décrit chaque étape BPMN à enrichir. Le bloc des données partagées définit les entités persistantes et leurs champs. Cette organisation rend la spécification lisible et vérifiable.

```
{
  "processId": "processus_demande",
  "schemaVersion": 1,
  "start": {
    "allowedRoles": ["DEMANDEUR"],
    "startActivities": ["saisir_demande"]
  },
  "monitoring": {
    "supervisorRoles": ["SUPERVISEUR", "ADMINISTRATEUR"]
  },
  "activities": {
    "saisir_demande": {
      "activityType": "MANUAL",
      "assignment": {},
      "input": {},
      "output": {},
      "visibility": {}
    }
  },
  "sharedData": {
    "entities": []
  }
}
```

**Listing III.1 –** *Structure simplifiée de la spécification spec-v1*

Pour que cette spécification puisse être exploitée par un générateur, sa structure ne doit pas rester implicite. Dans ProOWeb, *spec-v1* est considérée comme un contrat d'entrée : certains champs sont obligatoires, d'autres sont optionnels, et plusieurs valeurs appartiennent à des listes contrôlées. Cette formalisation évite que deux utilisateurs décrivent le même besoin avec des formes différentes et permet au générateur de produire des diagnostics précis avant la génération.

**TABLE V** – *Définition formelle des principaux champs de la spécification spec-v1*

| Champ | Type attendu | Contrainte principale |
|---|---|---|
| `processId` | Chaîne | Identifiant stable du processus ; il doit correspondre au processus BPMN importé. |
| `schemaVersion` | Entier | Version du format de spécification ; pour ce travail, la valeur attendue est `1`. |
| `start.allowedRoles` | Liste de rôles | Chaque rôle doit appartenir au référentiel de rôles de l'application. |
| `activities` | Objet indexé | Chaque clé doit être l'identifiant d'une activité BPMN enrichie. |
| `activityType` | Enumération | Valeurs autorisées : `MANUAL` ou `AUTOMATIC`. |
| `assignment.candidateRoles` | Liste de rôles | Une tâche manuelle doit toujours avoir au moins un rôle capable de la traiter. |

### III.4.2. Métadonnées des activités

Les métadonnées d'activité indiquent comment une étape BPMN doit être prise en charge dans l'application. Une activité manuelle est destinée à être traitée par un utilisateur. Elle doit donc préciser les rôles candidats, la stratégie d'affectation, les données visibles, les sorties attendues et les règles de complétion. Une activité automatique est exécutée par le système. Elle doit alors indiquer le gestionnaire à invoquer, son type, son mode de déclenchement et sa configuration.

```
"valider_demande": {
  "activityType": "MANUAL",
  "assignment": {
    "candidateRoles": ["VALIDATEUR"],
    "mode": "CLAIM",
    "strategy": "ROLE_BASED"
  },
  "input": {
    "sources": ["PROCESS_CONTEXT", "PREVIOUS_ACTIVITY"]
  },
  "output": {
    "storage": "BOTH",
    "mappings": []
  },
  "visibility": {
    "activityViewerRoles": ["VALIDATEUR", "SUPERVISEUR"],
    "dataViewerRoles": ["VALIDATEUR"]
```

```
18    }
19 }
```

**Listing III.2 –** *Exemple d'activité enrichie dans spec-v1*

Cet exemple montre le passage du processus vers l'application. L'identifiant `valider_demande` renvoie à une activité BPMN. Les rôles candidats orientent l'affectation de la tâche. Les sources d'entrée indiquent les données disponibles pour le formulaire. La stratégie de stockage précise où seront conservées les sorties. Les règles de visibilité distinguent les acteurs qui peuvent voir l'activité de ceux qui peuvent accéder aux données.

### III.4.3. Affectation des tâches et sécurité

L'affectation des tâches est un point central dans une application web métier. Une tâche doit être proposée aux bons utilisateurs, selon les responsabilités prévues par l'organisation. Dans ProOWeb, cette dimension est décrite par les rôles candidats, le mode d'affectation et la stratégie d'assignation.

**TABLE VI –** *Exemples de stratégies d'affectation*

| Stratégie | Interprétation dans ProOWeb |
|---|---|
| `ROLE_QUEUE` | La tâche est visible dans une file associée aux rôles candidats. |
| `ROLE_BASED` | La tâche est attribuée ou réclamée selon les rôles déclarés dans la spécification. |
| `SUPERVISOR_ONLY` | La tâche est réservée aux rôles de supervision. |
| `SUPERVISOR_THEN_ANCESTORS` | La tâche peut être escaladée dans une chaîne hiérarchique lorsque la règle locale ne suffit pas. |

### III.4.4. Entrées, sorties et données partagées

Une application web métier ne se contente pas d'enchaîner des tâches. Elle manipule des informations. ProOWeb distingue les données propres à une instance (décrivant l'état d'une exécution particulière) et les données partagées (représentant des informations métier réutilisables, persistées et consultables au-delà d'une instance).

Les entrées d'une activité peuvent provenir du contexte du processus, d'une activité précédente, d'une donnée partagée ou d'une source externe. Les sorties peuvent être stockées dans l'instance, dans les données partagées ou dans les deux espaces.

Le bloc `sharedData` définit les entités partagées. Ces entités peuvent ensuite être projetées vers des classes de domaine, des repositories, des migrations de schéma et des métadonnées de formulaires. Lorsque les mappings de sortie indiquent qu'une activité produit une donnée partagée, ProOWeb peut établir le lien entre la tâche, le champ produit et la structure persistante cible.

**TABLE VII –** *Correspondance entre informations d'entrée et application générée*

| **Information d'entrée** | **Projection intermédiaire** | **Artefact généré** |
|---|---|---|
| Activité BPMN | Activité du *runtime contract* | Tâche runtime, vue de traitement et action de complétion. |
| Flux BPMN | Transitions et liens entrants/sortants | Logique d'orchestration et progression d'instance. |
| Rôles candidats | Politique d'affectation | Liste de tâches, assignation et contrôle d'accès. |
| Sources d'entrée | Lignage des données | Pré-remplissage de formulaires et résolution de contexte. |
| Mappings de sortie | Stratégie de stockage | Mise à jour de l'instance, des données partagées ou des deux. |
| Règles de visibilité | Droits de lecture activité/donnée | Filtrage applicatif et adaptation de l'interface. |
| Données partagées | Entités, champs et relations | Modèles persistants, migrations et métadonnées. |

### III.4.5. Règles de visibilité

La visibilité ne se réduit pas à l'affectation. Un utilisateur peut être autorisé à consulter une instance sans être autorisé à traiter la tâche courante. De même, un superviseur peut voir l'état d'un processus sans accéder à toutes les données sensibles produites par les activités. ProOWeb distingue donc la visibilité des activités et la visibilité des données.

Cette distinction permet de générer des filtres plus fins côté services applicatifs et des interfaces plus adaptées côté utilisateur. Les services applicatifs peuvent masquer certaines tâches ou certaines données selon le rôle de l'utilisateur. Les composants d'interface peuvent également adapter les informations affichées sans exposer inutilement des données sensibles.

## III.5. Règles de transformation et génération de l'application web

Les règles de transformation déterminent la manière dont le double modèle d'entrée devient une application web métier. Dans ProOWeb, la transformation suit deux moments principaux. La transformation modèle-à-modèle construit le modèle applicatif intermédiaire. La transformation modèle-à-texte produit les fichiers du projet cible. Ce découpage permet de vérifier les contrats avant l'écriture du code.

### III.5.1. Validation avant génération

La validation est une étape indispensable. Une erreur dans les modèles d'entrée peut se propager dans plusieurs fichiers générés : point d'accès applicatif, composant d'interface, transition, structure de données ou règle de sécurité. Le framework doit donc refuser ou signaler les situations ambiguës avant de produire le projet cible.

Les contrôles portent sur quatre dimensions. La dimension structurelle vérifie que les activités enrichies existent dans le BPMN et que les transitions forment un graphe exploitable. La dimension applicative vérifie que les activités manuelles disposent d'informations suffisantes pour être traitées par un utilisateur. La dimension données vérifie que les entrées, sorties et mappings référencent des structures connues. La dimension sécurité vérifie la cohérence des rôles utilisés pour l'exécution, la supervision et la visibilité.

**TABLE VIII –** *Exemples de contrôles réalisés avant génération*

| **Dimension** | **Règle contrôlée** | **Risque évité** |
|---|---|---|
| Structure BPMN | Toute activité enrichie doit exister dans le modèle BPMN. | Artefacts sans activité correspondante. |
| Sécurité | Une tâche manuelle doit posséder au moins un rôle candidat. | Tâche exposée sans responsabilité claire. |
| Données | Un mapping de sortie doit cibler une donnée connue. | Perte ou mauvaise persistance des données. |
| Workflow | Une transition générée doit correspondre à un chemin possible. | Divergence entre modèle et runtime. |
| Interface | Les champs affichés doivent provenir d'une source déclarée. | Formulaire incohérent avec le contexte. |

Ces contrôles doivent produire des diagnostics compréhensibles. L'utilisateur du framework doit pouvoir identifier l'activité, la règle ou la donnée responsable de l'erreur.

### III.5.2. Transformation modèle-à-modèle

La transformation modèle-à-modèle prend en entrée le BPMN et *spec-v1*. Elle commence par normaliser les activités. Chaque activité reçoit un type, des rôles candidats, une politique d'affectation, des entrées, des sorties et une visibilité. Les activités automatiques reçoivent en plus un gestionnaire, un type de tâche automatique, un mode de déclenchement et une configuration.

À partir du BPMN, ProOWeb détermine les transitions entre activités, les liens entrants et les liens sortants. Les activités de départ peuvent être configurées explicitement ou inférées à partir du graphe. Cette étape permet au runtime de savoir comment démarrer une instance et comment la faire progresser.

Les mappings d'entrée et de sortie sont parcourus afin de relier les activités, les champs produits, les entités partagées et les sources utilisées. Cette représentation permet de comprendre comment une information circule dans le processus et comment elle se matérialise dans l'application.

**Construction du modèle applicatif intermédiaire**

Le modèle applicatif intermédiaire regroupe uniquement les informations nécessaires à la génération : déroulement du processus, rôles, données, formulaires et traces des artefacts produits. Il sert de contrat entre l'analyse du BPMN, l'enrichissement *spec-v1* et les gabarits de génération.

**TABLE IX –** *Structures principales du modèle pivot ProOWeb*

| Structure conservée | Utilité dans ProOWeb |
|---|---|
| `ProcessModel` | Garde la référence au processus BPMN, à ses activités et à ses transitions. |
| `ActivityModel` | Transforme chaque activité BPMN enrichie en unité exploitable par l'application. |
| `RolePolicy` | Regroupe les rôles, règles d'affectation, droits d'accès et restrictions de visibilité. |
| `FormModel` | Décrit les champs affichés, les sorties produites et les validations des formulaires. |
| `RuntimeContract` | Fournit au moteur d'exécution les états, transitions et actions disponibles. |
| `DataContract` | Prépare les entités, champs, mappings et règles de stockage des données. |
| `ArtifactTrace` | Relie les éléments du modèle aux fichiers générés afin de contrôler les régénérations. |

### III.5.3. Transformation modèle-à-texte

La transformation modèle-à-texte produit les fichiers de l'application. Les gabarits reçoivent les éléments du modèle pivot et les projettent dans les artefacts cibles. Le passage vers le texte transforme alors des structures abstraites en vues, composants, services, règles d'accès, structures de données et fichiers de configuration.

La qualité de cette transformation dépend de la précision des gabarits. Un gabarit ne doit pas contenir des décisions métier cachées. Il doit traduire les décisions déjà prises dans le modèle pivot. Par exemple, si une activité est réservée au rôle `VALIDATEUR`, le gabarit d'accès applicatif doit utiliser cette information pour produire le contrôle d'accès correspondant. Si une entité possède une contrainte de non-nullité, le gabarit de persistance et le formulaire associé doivent en tenir compte.

Le moteur M2T doit également conserver une organisation déterministe. À modèle identique, il doit produire la même structure de projet, les mêmes noms d'artefacts et les mêmes emplacements de fichiers. Cette propriété facilite la comparaison entre deux générations, l'identification des changements et la synchronisation avec un gestionnaire de versions.

### III.5.4. Exemple de traçabilité de bout en bout

La traçabilité est illustrée ici sur l'activité BPMN `valider_demande`. L'intérêt n'est pas de détailler séparément chaque couche technique, mais de montrer comment une intention métier reste reconnaissable jusqu'aux éléments produits par ProOWeb.

**TABLE X –** *Exemple de traçabilité d'une activité BPMN vers les artefacts générés*

| Niveau suivi | Interprétation dans ProOWeb |
|---|---|
| BPMN | La tâche `valider_demande` donne le point de passage métier et ses liens avec le reste du processus. |
| *spec-v1* | L'activité reçoit les précisions absentes du diagramme : rôle autorisé, données utiles, sorties attendues et visibilité. |
| Modèle pivot | ProOWeb rassemble ces informations dans une représentation stable, utilisable par les règles de génération. |
| Artefacts générés | Les fichiers produits matérialisent la même activité : écran de traitement, action de validation, contrôle d'accès et mise à jour des données. |
| Manifeste | Les fichiers rattachés à cette activité restent identifiables afin de maîtriser les futures régénérations. |

### III.5.5. Règles de projection vers les artefacts

Les règles de projection définissent comment chaque élément du modèle pivot est traduit dans l'application cible. Elles ne sont pas de simples conventions de nommage. Elles indiquent la couche de destination et la responsabilité de l'artefact produit.

**TABLE XI –** *Exemples de règles de projection vers l'application générée*

| **Élément du modèle pivot** | **Artefact généré** | **Couche cible** |
|---|---|---|
| Activité manuelle | Formulaire, page d'activité, structure d'entrée | Interface utilisateur et couche applicative. |
| Activité automatique | Opération système ou service applicatif | Couche applicative. |
| Rôle candidat | Règle d'accès et filtre de tâche | Sécurité, services applicatifs et interface. |
| Transition du workflow | Méthode de progression et changement d'état | Runtime du processus. |
| Entité partagée | Structure métier, stockage, migration | Domaine et persistance. |
| Mapping de sortie | Service de sauvegarde et projection cible | Application et persistance. |

Cette projection maintient la cohérence entre le modèle et le projet produit. Le générateur ne crée pas seulement des fichiers ; il garde le lien entre chaque élément généré et le besoin métier qui l'a rendu nécessaire. Ce lien rend le code plus compréhensible lorsqu'il faut l'adapter ou le régénérer.

**TABLE XII –** *Inventaire des artefacts générés par ProOWeb*

| **Besoin couvert** | **Rôle dans l'application produite** |
|---|---|
| Exécuter le processus | Démarrer une instance, proposer les tâches, appliquer les transitions et enregistrer l'avancement. |
| Traiter les activités | Donner aux acteurs les vues, formulaires et actions nécessaires pour accomplir leurs tâches. |
| Conserver les informations | Garder les données de l'instance, les données partagées et l'historique des actions importantes. |
| Contrôler les accès | Appliquer les rôles candidats, les règles de visibilité et les restrictions liées à chaque activité. |
| Suivre la génération | Identifier les fichiers produits, signaler les changements et protéger les adaptations manuelles. |

## III.6. Préservation du code personnalisé et synchronisation

Une difficulté importante de la génération d'application concerne l'évolution du projet après une première génération. Dans un contexte réel, les développeurs ajoutent des règles particulières, adaptent les interfaces, complètent des services ou intègrent des dépendances spécifiques. Si chaque nouvelle génération écrase ces modifications, le framework devient difficile à utiliser. ProOWeb traite donc la régénération comme une opération d'ingénierie, et non comme un simple remplacement de fichiers.

Cette préoccupation est bien identifiée dans les travaux sur la cohabitation entre code généré et code écrit à la main. Greifenberg et al. comparent plusieurs mécanismes d'intégration et montrent que le choix d'une stratégie influence directement la maintenabilité des systèmes générés [47].

### III.6.1. Problème de la régénération

La régénération apparaît lorsqu'un processus évolue, lorsqu'une activité est ajoutée, lorsqu'une règle de données change. Elle est nécessaire pour maintenir l'application alignée sur les modèles. Elle est également risquée, car elle peut remplacer des fichiers modifiés par les développeurs.

Le problème ne se limite pas à la perte de code. Une régénération non contrôlée peut produire des conflits de fichiers, des incohérences entre la partie applicative et l'interface, la disparition de règles spécifiques, un changement non documenté dans la sécurité ou une rupture des tests. ProOWeb distingue donc les fichiers gérés par le générateur et les fichiers non gérés. Les fichiers gérés sont suivis dans le manifeste `.prooweb-managed.json`. Les fichiers non gérés restent hors du contrôle du générateur.

### III.6.2. Manifeste et contrat de propriété

Le manifeste `.prooweb-managed.json` décrit les fichiers produits par ProOWeb. Pour chaque fichier, il peut conserver le chemin, la catégorie, les propriétaires logiques et une empreinte. Cette information permet de savoir si un fichier correspond toujours à la dernière version générée ou s'il a été modifié manuellement.

Ce manifeste définit un contrat de propriété. Un fichier généré et non modifié peut être remplacé lors d'une nouvelle génération. Un fichier généré mais modifié doit être traité avec prudence. Un fichier non géré ne doit pas être écrasé. Cette règle donne au développeur une garantie pratique : ProOWeb ne considère pas le projet cible comme un répertoire jetable.

### III.6.3. Préservation côté logique applicative

Côté logique applicative, la préservation est particulièrement importante, car le code généré porte les API, la sécurité, la persistance et le runtime. ProOWeb limite le risque en produisant une architecture modulaire. Les éléments issus du modèle sont générés dans des couches précises, tandis que les adaptations locales peuvent être placées dans des modules ou points d'extension séparés.

Cette stratégie rejoint le principe du *Generation Gap Pattern*, souvent utilisé pour séparer les classes générées des classes écrites manuellement [47]. Le générateur peut produire un squelette ou une classe de base, tandis que le développeur ajoute un comportement spécifique dans une classe séparée. Lors d'une régénération, la partie contrôlée par ProOWeb peut évoluer sans supprimer les extensions manuelles.

La préservation ne doit toutefois pas servir à contourner le modèle. Si une modification concerne le workflow, les rôles, les données structurantes ou les règles de visibilité, elle devrait être reportée dans le BPMN ou dans *spec-v1*. Les extensions manuelles doivent plutôt couvrir les détails d'intégration, les règles particulières ou les adaptations difficiles à généraliser.

### III.6.4. Préservation côté interface utilisateur

Côté interface utilisateur, la personnalisation concerne souvent les libellés, les parcours utilisateur, les composants visuels, les contrôles d'ergonomie et certaines intégrations. ProOWeb génère les composants liés au processus, mais l'équipe doit pouvoir adapter l'expérience utilisateur sans perdre ses modifications lors d'un redéploiement.

La stratégie retenue repose sur la composition, la configuration et les overrides de templates. Les composants générés peuvent être encapsulés par des composants propres au projet. Les règles d'override déclarées peuvent remplacer, préfixer, suffixer ou modifier un bloc de contenu généré. Cette personnalisation est réappliquée lors des générations suivantes, ce qui réduit les modifications répétées directement dans les fichiers produits.

### III.6.5. Cycle de synchronisation itératif

Le cycle de synchronisation décrit la manière dont ProOWeb accompagne les évolutions successives du modèle. Une première génération produit l'application de base. Les développeurs peuvent ensuite ajouter des extensions, adapter certains modules ou déclarer des overrides. Lorsque le BPMN ou *spec-v1* évolue, ProOWeb

reconstruit un snapshot de génération, compare l'état attendu avec l'état du projet, applique les règles de migration et produit un rapport.

Ce cycle transforme la génération en activité continue. L'application n'est pas simplement créée une fois ; elle peut être régénérée, migrée et adaptée. La synchronisation repose sur trois traces : le manifeste des fichiers gérés, les sauvegardes créées lors des conflits et les rapports de migration. Ces éléments permettent de comprendre ce qui a changé et de revenir sur une modification si nécessaire.

**TABLE XIII –** *Mécanismes de préservation et de synchronisation dans ProOWeb*

| **Mécanisme** | **Rôle** | **Effet attendu** |
|---|---|---|
| Manifeste | Suivre les fichiers générés, leurs catégories et leurs empreintes. | Identifier les fichiers que ProOWeb peut gérer. |
| Migration contrôlée | Comparer l'état attendu et l'état réel du projet. | Éviter l'écrasement silencieux du code personnalisé. |
| Sauvegardes | Copier les fichiers conflictuels avant remplacement. | Permettre la récupération des adaptations manuelles. |
| Template overrides | Réappliquer des personnalisations déclarées. | Réduire les modifications manuelles répétées. |
| Rapports | Décrire les créations, mises à jour, conflits et sauvegardes. | Rendre l'évolution auditable et compréhensible. |

## III.7. Conclusion

Ce chapitre a présenté l'approche proposée et sa concrétisation dans le framework ProOWeb, depuis les modèles d'entrée jusqu'aux artefacts générés. Le BPMN décrit le déroulement du processus, *spec-v1* apporte les informations applicatives nécessaires et le modèle pivot organise ces données avant la génération.

L'approche met l'accent sur deux points : maintenir la traçabilité entre le modèle et le code, puis permettre l'évolution contrôlée de l'application générée. Le chapitre suivant présente la mise en oeuvre technique de cette proposition et son application à un cas d'utilisation.

# IV

CHAPITRE

# MISE EN OEUVRE ET ÉVALUATION DE PROOWEB

SOMMAIRE

IV.1 - Introduction . . . . . 55
IV.2 - Environnement de mise en oeuvre . . . . . 56
IV.3 - Implémentation de la chaîne de génération . . . . . 58
IV.4 - Cas d'étude organisationnel . . . . . 62
IV.5 - Évaluation de ProOWeb . . . . . 68
IV.6 - Conclusion . . . . . 73

## IV.1. Introduction

Le chapitre précédent a présenté le framework proposé pour générer une application web métier à partir d'un modèle BPMN complété par une spécification applicative. Il a notamment défini l'architecture générale de ProOWeb, le rôle de *spec-v1*, les transformations modèle-à-modèle et modèle-à-texte, ainsi que les mécanismes de préservation du code personnalisé. Nous passons maintenant de cette description d'ensemble à sa mise en oeuvre, en observant comment ProOWeb produit une application métier destinée à une organisation de commerce électronique.

Le chapitre suit donc le chemin emprunté par le framework : initialisation du projet, modélisation ou importation des processus, validation des spécifications enrichies, déploiement des contrats runtime et data, génération des artefacts backend/frontend et vérification des parcours obtenus. Cette progression permet de relier le modèle conceptuel du chapitre III au résultat produit dans l'application générée.

## IV.2. Environnement de mise en oeuvre

### IV.2.1. Objectif de réalisation

La mise en oeuvre de ProOWeb a consisté à rendre opérationnel le framework décrit au chapitre précédent. Les blocs conceptuels présentés dans l'architecture de la section III.3 sont ici traduits en composants réalisés : un noyau de génération, des mécanismes de validation, des contrats intermédiaires, des gabarits et un projet applicatif cible. Le framework reçoit des informations métier structurées, vérifie leur cohérence, construit des contrats intermédiaires et produit une base applicative exploitable. Cette base contient les éléments nécessaires pour exécuter un processus, afficher les tâches, appliquer les rôles, manipuler les données et suivre les fichiers générés.

Dans ce travail, ProOWeb n'est donc pas présenté comme une simple maquette. Il sert à vérifier si le passage du modèle BPMN vers une application web métier peut être automatisé tout en gardant une trace claire entre le besoin métier et les artefacts produits.

### IV.2.2. Choix technologiques

Le générateur a été séparé de l'application produite. Cette séparation reprend la distinction établie au chapitre III entre le noyau de génération et l'application web générée. ProOWeb s'exécute comme une application Node.js. Il fournit une interface de configuration, des services de gestion du workspace et des modules spécialisés dans la modélisation de processus : catalogue, validation de *spec-v1*, simulation, déploiement, génération des contrats runtime et gestion des fichiers produits.

L'application générée, quant à elle, repose sur Spring Boot pour le backend, React/Vite pour l'interface web et PostgreSQL comme base de données cible. Le workspace généré pour le cas d'étude porte le nom E-commerce Application. Il active notamment la modélisation de processus, la hiérarchie organisationnelle, les notifications, Liquibase, Swagger en profil de développement, les scripts de lancement et les tests frontend Cypress. Ce choix technique n'est donc pas seulement déclaratif : il correspond à une arborescence générée, à des fichiers de configuration et à des scripts de vérification présents dans le projet.

**TABLE XIV –** *Principaux choix de mise en oeuvre de ProOWeb*

| Éléments | Choix retenu | Rôle dans ProOWeb |
|---|---|---|
| Noyau du framework | Node.js / JavaScript | Lire le workspace, importer les modèles, valider *spec-v1*, construire les contrats et déclencher le déploiement. |
| Interface de configuration | Pages web ProOWeb | Initialiser le projet, accéder au tableau de bord et piloter certaines opérations de génération. |
| Backend généré | Spring Boot modulaire | Porter les modules système, organisation, sécurité, runtime de processus et ressources métier générées. |
| Frontend généré | React/Vite | Fournir les vues de travail, notamment le *Process Runtime Workbench* utilisé dans les tests. |
| Base de données cible | PostgreSQL avec Liquibase | Préparer la persistance et les scripts de migration nécessaires au projet produit. |
| Suivi des fichiers | Manifeste de génération | Identifier les fichiers générés et soutenir les régénérations contrôlées. |
| Vérification | Scripts et tests Cypress | Vérifier les parcours frontend liés aux processus déployés. |

### IV.2.3. Organisation réalisée

ProOWeb est organisé autour de deux espaces qui gardent chacun une responsabilité claire. D'un côté, le framework contient les services de lecture des modèles, les validations, la construction des contrats, les gabarits et les mécanismes de suivi. De l'autre, le projet cible regroupe les fichiers applicatifs produits à partir des modèles et destinés à être exécutés, testés puis adaptés.

Cette séparation se voit directement dans l'arborescence du projet. Les modules du framework sont placés dans `ProOWeb/src/lib` et `ProOWeb/src/services`. Les artefacts générés sont présents dans le workspace applicatif, avec un backend Spring Boot, un frontend React et des ressources de processus rangées dans le dossier `resources/processes` du module applicatif. Pour chaque processus déployé, ProOWeb conserve le BPMN, une spécification transformée, un contrat runtime et un contrat de données. Cette organisation rend le résultat plus lisible : le modèle d'entrée reste consultable, tandis que les contrats générés expliquent comment le modèle est projeté vers l'application.

**TABLE XV** – *Organisation fonctionnelle réalisée dans ProOWeb*

| Bloc | Responsabilité principale |
|---|---|
| Workspace généré | Conserver la configuration de l'application e-commerce, les scripts et les modules générés. |
| Catalogue de processus | Enregistrer les modèles, les versions, les statuts de validation et les informations de déploiement. |
| Validation *spec-v1* | Refuser ou signaler les incohérences entre le BPMN, les rôles, les données et les activités enrichies. |
| Contrats runtime et data | Préparer une représentation stable des activités, transitions, rôles, données partagées et règles d'accès. |
| Ressources backend | Déposer les fichiers `v1.bpmn`, `v1.json`, `v1.runtime.json` et `v1.data.json` par processus. |
| Manifeste | Garder la mémoire des fichiers générés afin de contrôler les évolutions ultérieures. |

## IV.3. Implémentation de la chaîne de génération

La figure 28 présente la chaîne de génération mise en oeuvre dans ProOWeb. Elle reprend les étapes conceptuelles décrites dans la section III.5 : lecture des modèles, validation, construction d'une représentation intermédiaire, projection vers les artefacts et suivi de la génération. Elle montre ainsi le passage des modèles d'entrée vers les contrats intermédiaires, puis vers les artefacts applicatifs et le manifeste de traçabilité utilisé lors des régénérations.

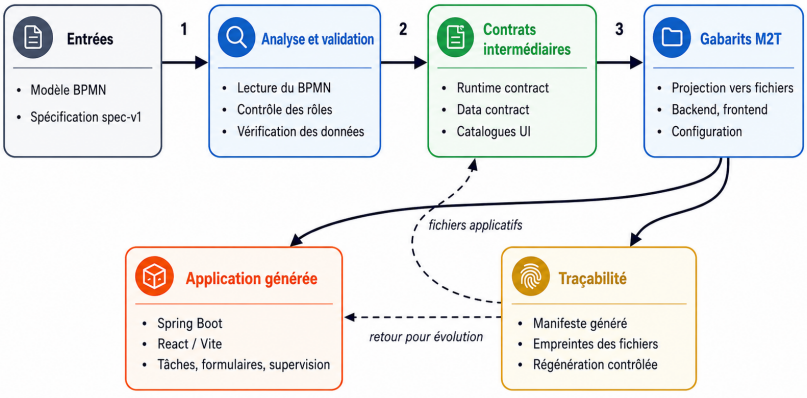



**FIGURE 28** – *Chaîne de génération de ProOWeb*

### IV.3.1. Initialisation du projet cible

La génération commence par l'initialisation du projet cible. Cette étape fixe l'identité de l'application à produire : nom du projet, espace de travail, package de base, options d'authentification, profils d'exécution et paramètres généraux. Dans le cas d'étude, cette initialisation a produit le workspace E-commerce Application, avec le package de base `com.prooweb.generated`, un backend Spring Boot, un frontend React et une base de données cible PostgreSQL.

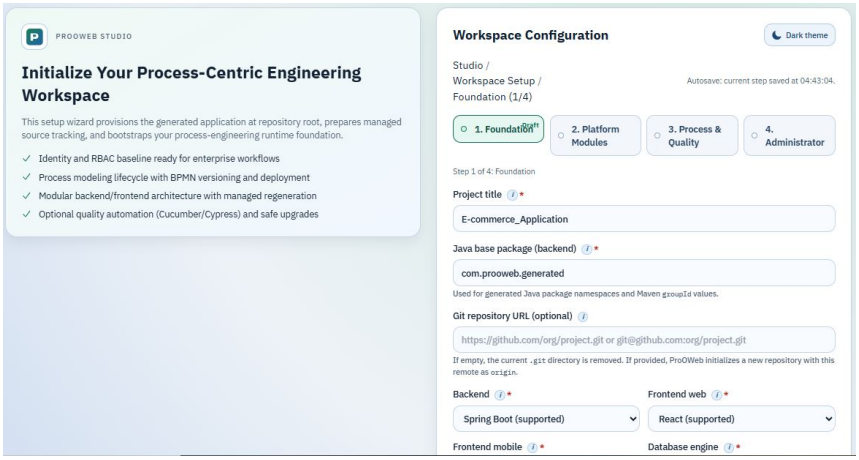



**FIGURE 29 –** *Initialisation du projet* `E-commerce_Application`

Cette étape prépare une application structurée, équipée de scripts de compilation, de vérification, de test et de lancement par profil. Le workspace contient ainsi les scripts `build-all`, `test-all`, `verify-all` et `start-profile`, ainsi que les raccourcis NPM qui permettent de piloter ces opérations depuis la racine du projet.

### IV.3.2. Gestion des modèles de processus

Après l'initialisation, ProOWeb prend en charge les modèles BPMN et les spécifications *spec-v1*. Le BPMN garde le rôle de support de workflow. La spécification enrichie ajoute les informations que le modèle ne porte pas naturellement : rôles candidats, règles d'affectation, entrées, sorties, données partagées, visibilité et stratégies de stockage.

Pour le cas d'étude, les trois BPMN et les trois fichiers de spécification conservés dans le chapitre 4 sont importés dans ProOWeb par un script dédié. Ce script lit les sources, normalise certaines informations, transforme la spécification de travail vers la forme attendue par ProOWeb, puis lance la validation stricte avec le catalogue des tâches automatiques. Lorsque le modèle est recevable, une version est créée ou ajoutée dans le catalogue, simulée, puis déployée. Cette étape donne une valeur particulière au cas d'étude : les processus présentés dans le mémoire ne res-

tent pas au niveau du dessin, ils deviennent des versions de processus manipulées par le framework.

### IV.3.3. Validation et construction des contrats

La validation est le premier point de contrôle sérieux de la chaîne. Elle met en oeuvre les contrôles introduits dans le chapitre précédent : cohérence entre BPMN et *spec-v1*, présence des rôles nécessaires, validité des données et correspondance entre activités et artefacts à produire.

Lorsque ces contrôles sont satisfaits, ProOWeb construit des contrats intermédiaires. Ces contrats correspondent à la matérialisation du modèle pivot présenté au chapitre III. Dans le workspace généré, ils apparaissent sous la forme d'un catalogue runtime, d'un catalogue de données, d'un catalogue de tâches automatiques et de contrats propres à chaque processus. Ils ne sont pas encore le comportement exécuté par un utilisateur, mais ils donnent au générateur une base stable pour produire les vues, les règles de progression et les données partagées.

**TABLE XVI –** *Contrats produits avant la génération des fichiers*

| Contrat | Utilisation dans ProOWeb |
|---|---|
| *Runtime contract* | Décrire les processus, activités, transitions, statuts et actions disponibles pendant l'exécution. |
| *Data contract* | Organiser les données partagées, les champs, les sources et les mappings issus des activités. |
| Catalogue des formulaires | Définir les champs affichés, les champs produits et les contraintes des activités manuelles. |
| Catalogue d'interface | Préparer les vues utiles : listes de tâches, actions, suivi et règles d'affichage. |
| Registre des processus | Conserver les processus disponibles, leurs versions et leurs métadonnées d'exploitation. |

### IV.3.4. Génération des artefacts applicatifs

La génération transforme les contrats en fichiers organisés dans le projet cible. Elle concrétise les règles de projection décrites au chapitre précédent : une activité, un rôle, une transition ou une donnée partagée sont traduits en artefacts applicatifs cohérents. Une activité BPMN enrichie ne devient donc pas seulement une ligne dans un fichier de configuration. Elle donne lieu à un ensemble cohérent : une action de traitement, un formulaire, des contrôles d'accès, des transitions de processus et des traces exploitables dans l'historique.

Le résultat attendu est une base applicative cohérente, et non une collection de fichiers isolés. Les artefacts produits sont regroupés autour de besoins simples : exécuter un processus, faire traiter les tâches, conserver les informations, contrôler les accès et suivre l'évolution du projet généré.

```
E-commerce_Application/
  src/
    backend/
      springboot/
        common/
        organization/
        system/
        prooweb-application/
          src/main/resources/processes/
    frontend/
      web/react/
        src/
        cypress/e2e/
  deployment/
    docker/
  build-all.ps1
  test-all.ps1
  verify-all.ps1
  start-profile.ps1
  .prooweb-managed.json
```

**Listing IV.1 –** *Organisation simplifiée du workspace généré*

### IV.3.5. Préservation et régénération contrôlée

ProOWeb met en place un manifeste des fichiers générés. Ce choix prolonge les mécanismes de préservation et de synchronisation discutés dans la section III.6. Le manifeste garde la trace des chemins, des catégories et des empreintes associées aux fichiers produits. Lorsqu'une nouvelle génération est lancée, le framework peut comparer l'état attendu avec l'état présent dans le projet cible.

Ce mécanisme répond à une difficulté réelle : une application générée n'est pas figée. Une équipe peut modifier un libellé, ajouter une règle locale, connecter un service externe ou adapter une vue. La régénération ne doit donc pas effacer ces interventions sans contrôle. ProOWeb cherche à garder le lien entre modèle et code tout en respectant les adaptations humaines.

```
{
  "generatedAt": "2026-06-25",
  "files": [
    {
      "path": "src/backend/.../processes/commande-client/v1.runtime.json",
```

```
      "source": "commande-client:v1",
      "category": "process-runtime-contract",
      "managed": true
    },
    {
      "path":
   "src/frontend/web/react/cypress/e2e/process-commande-client-v1.cy.js",
      "source": "commande-client:v1",
      "category": "frontend-e2e-test",
      "managed": true
    }
  ]
}
```

**Listing IV.2 –** *Extrait simplifié du manifeste de génération*

## IV.4. Cas d'étude organisationnel

### IV.4.1. Présentation de l'organisation retenue

Le cas d'étude retenu concerne une organisation de commerce électronique. Cette organisation vend des produits à travers une boutique en ligne et s'appuie sur plusieurs pôles internes : prise en charge des commandes, gestion du stock, préparation logistique, service client et supervision commerciale. Son activité ne se résume donc pas à l'affichage d'un catalogue. Elle implique des décisions, des contrôles, des validations et des échanges entre acteurs avant qu'une commande soit clôturée ou qu'une réclamation soit traitée.

Ce terrain est intéressant pour ProOWeb parce que les processus ne sont pas isolés. Une commande client peut consommer du stock ; une rupture peut déclencher un réapprovisionnement ; une réclamation peut concerner une commande déjà livrée et nécessiter une vérification commerciale ou logistique. Le cas d'étude permet ainsi de vérifier si une même application générée peut regrouper plusieurs workflows tout en conservant une cohérence métier autour des clients, des produits, des commandes et des mouvements de stock.

Ce cas d'étude sert donc de terrain d'application aux choix de conception présentés dans le chapitre III. Les processus retenus permettent d'observer l'utilisation conjointe du BPMN, de la spécification *spec-v1*, des contrats intermédiaires, des règles de projection et du mécanisme de traçabilité.

### IV.4.2. Acteurs et responsabilités

Les acteurs retenus correspondent aux responsabilités courantes d'une organisation de commerce électronique.

**TABLE XVII –** *Acteurs du cas d'étude e-commerce*

| Acteur | Responsabilité principale | Interaction avec l'application générée |
|---|---|---|
| Client | Passer une commande ou déposer une réclamation de produit | Renseigne les informations nécessaires et suit l'état de ses demandes. |
| Service commercial | Suivre les commandes et arbitrer les cas nécessitant une décision | Consulte les commandes, confirme certaines décisions et suit les exceptions. |
| Gestionnaire de stock | Vérifier les disponibilités, préparer les commandes et enregistrer les mouvements | Reçoit les tâches liées au stock, à la préparation et à la réception des produits. |
| Responsable achats | Valider les réapprovisionnements et suivre les demandes fournisseur | Examine les besoins de stock et autorise les commandes d'achat. |
| Service client | Qualifier les réclamations de produit et proposer une solution | Traite les demandes après-vente et renseigne les décisions prises. |
| Administrateur | Configurer les rôles, produits, catégories et paramètres généraux | Maintient les données de base utilisées par les processus générés. |

### IV.4.3. Processus retenus

Trois processus ont été retenus pour représenter le fonctionnement de cette organisation. La commande client couvre le parcours qui va de la soumission de la commande à sa préparation. Le réapprovisionnement du stock intervient lorsqu'un produit devient insuffisant ou lorsqu'un responsable anticipe une rupture. La réclamation de produit prend en charge les demandes formulées après une vente. Ces processus partagent des données et des acteurs, mais ils répondent à des objectifs opérationnels différents.

**TABLE XVIII –** *Processus métier retenus dans le cas d'étude*

| Processus | Objectif métier | Acteurs clés | Résultat observé |
|---|---|---|---|
| P1 : commande client | Recevoir, vérifier, préparer et clôturer une commande passée sur la boutique en ligne | Client, service commercial, gestionnaire de stock | Commande confirmée, préparée, annulée ou clôturée. |
| P2 : réapprovisionnement du stock | Détecter un besoin de stock, le faire valider et enregistrer la réception des produits | Gestionnaire de stock, responsable achats | Stock réapprovisionné ou demande refusée. |
| P3 : réclamation de produit | Traiter une réclamation liée à une commande et proposer une solution adaptée | Client, service client, service commercial, gestionnaire de stock | Réclamation clôturée avec décision tracée. |

### IV.4.4. Déroulement des scénarios

Le processus P1 commence lorsqu'un client valide une commande. L'application crée une instance associée au client, aux lignes de commande et aux produits concernés. Le service commercial peut vérifier les informations sensibles, puis le gestionnaire de stock contrôle la disponibilité et prépare la commande. Lorsque la préparation est terminée, la commande peut être marquée comme prête à expédier ou clôturée selon le scénario retenu.

Le processus P2 part d'un besoin de réapprovisionnement. Ce besoin peut être identifié par un gestionnaire de stock lorsqu'un seuil est atteint ou lorsqu'une préparation de commande révèle une quantité insuffisante. La demande est transmise au responsable achats, qui valide, refuse ou demande des précisions. En cas de validation, la réception des produits met à jour les informations de stock.

Le processus P3 intervient après la vente. Un client signale un problème sur un produit livré : défaut constaté, erreur de référence, quantité incorrecte ou description non conforme. Le service client qualifie la demande, vérifie les informations de commande et analyse le motif de réclamation. Lorsque la situation exige une vérification interne, le gestionnaire de stock contrôle la référence ou la disponibilité du produit. La décision finale est ensuite enregistrée et communiquée au client.

**TABLE XIX –** *Scénarios exécutés dans le cas pratique*

| Processus | Parcours vérifié | Ce que le parcours permet d'observer |
|---|---|---|
| P1 | Validation, vérification, préparation et clôture d'une commande | Génération des tâches de traitement, transitions de commande, contrôles de stock et statuts d'instance. |
| P2 | Détection du besoin, validation d'achat, réception et mise à jour du stock | Enchaînement entre demande interne, décision responsable et mouvement de stock. |
| P3 | Dépôt d'une réclamation, qualification, vérification et décision | Traçabilité après-vente, visibilité des rôles et conservation des décisions liées à la commande. |

### IV.4.5. Spécifications enrichies associées

Pour chaque processus, le modèle BPMN est complété par une spécification *spec-v1*. Cette spécification précise les informations qui ne sont pas naturellement portées par le modèle : rôles autorisés, données d'entrée, sorties produites, règles de stockage et règles de visibilité.

L'extrait suivant montre une activité manuelle du processus P1. Il ne représente pas toute la spécification ; il illustre le type d'information que ProOWeb exploite pour produire une tâche réellement traitable dans l'application générée.

```
{
  "activityId": "preparer_commande",
  "type": "manual",
  "candidateRoles": [
    "GESTIONNAIRE_STOCK"
  ],
  "inputs": [
    "commande.numero",
    "client.identite",
    "lignesCommande",
    "stock.disponibilite"
  ],
  "outputs": [
    "decisionPreparation",
    "referenceColis",
    "commentairePreparation"
  ],
  "visibility": {
    "activityViewerRoles": [
      "GESTIONNAIRE_STOCK",
      "SERVICE_COMMERCIAL"
```

```
    ],
    "dataViewerRoles": [
      "SERVICE_COMMERCIAL"
    ]
  },
  "storage": "BOTH"
}
```

**Listing IV.3 –** *Exemple simplifié d'activité enrichie pour le cas d'étude*

**TABLE XX –** *Synthèse des spécifications enrichies par processus*

| Processus | Rôles principaux | Données manipulées | Écrans générés |
|---|---|---|---|
| P1 | Client, service commercial, gestionnaire de stock | Client, commande, produits, disponibilités | Commande, vérification, préparation et suivi. |
| P2 | Gestionnaire de stock, responsable achats | Produit, seuil, quantité demandée, réception | Demande de réapprovisionnement, validation et réception. |
| P3 | Client, service client, service commercial, gestionnaire de stock | Commande, motif, produit concerné, décision | Réclamation, instruction, vérification et clôture. |

### IV.4.6. Application web métier générée

À partir des trois processus et de leurs spécifications, ProOWeb produit une application web métier unique pour l'organisation de commerce électronique. L'utilisateur ne voit pas trois applications séparées. Il accède à un même environnement dans lequel les commandes, les besoins de stock, les réclamations, les tâches et les vues de suivi sont organisés selon son rôle. Les images suivantes présentent quelques interfaces de l'application générée.

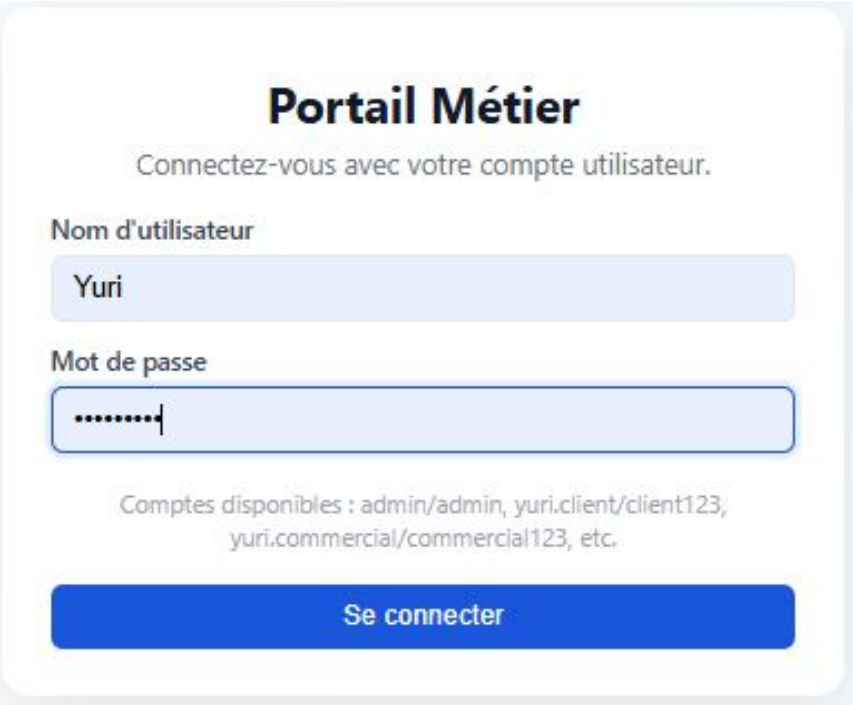



**FIGURE 30 –** *Portail de connexion à l'application* `E-commerce_Application`

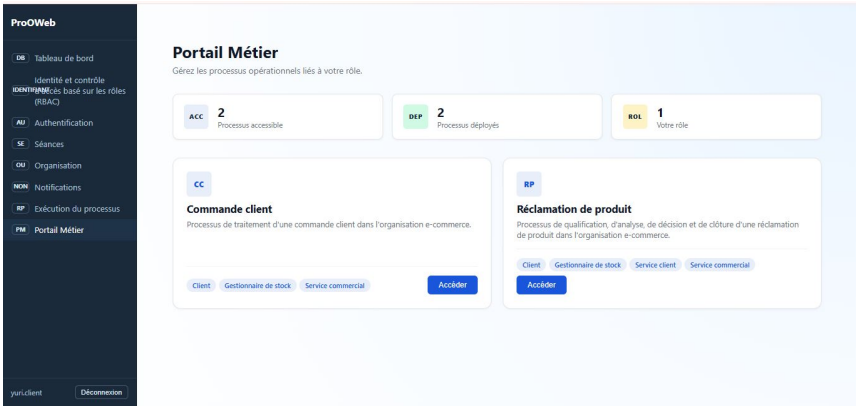



**FIGURE 31 –** *Tableau de bord pour un utilisateur connecté avec le rôle* `CLIENT`

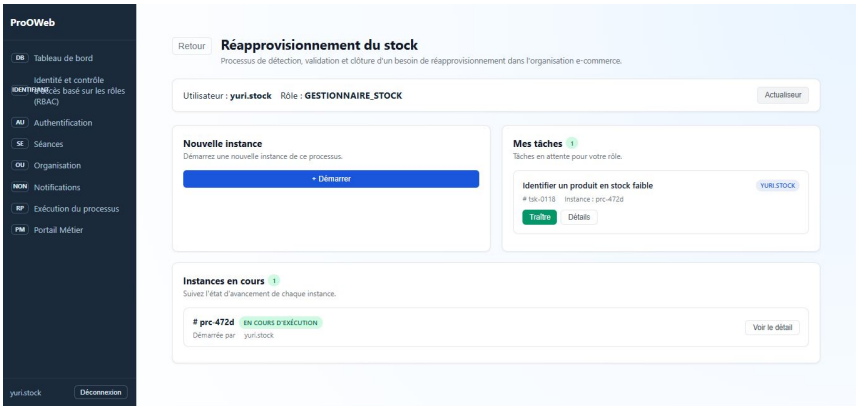



**FIGURE 32 –** *Tableau de bord pour un utilisateur connecté avec le rôle* `GESTIONNAIRE_STOCK`

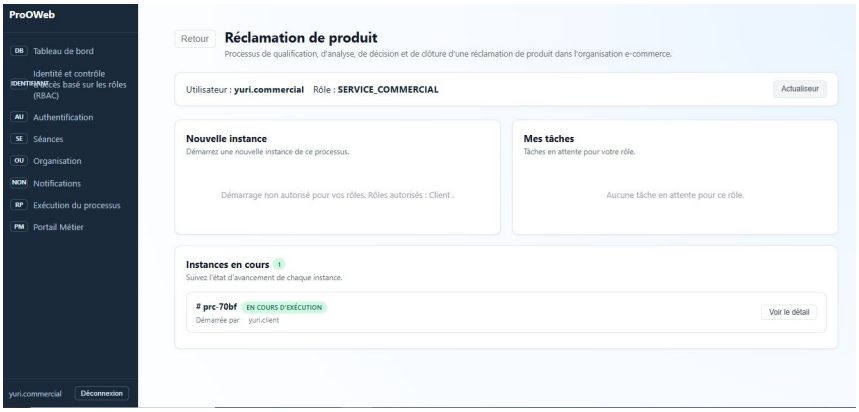



**FIGURE 33 –** *Tableau de bord pour un utilisateur connecté avec le rôle* `SERVICE_COMMERCIAL`

## IV.5. Évaluation de ProOWeb

### IV.5.1. Méthode d'évaluation

L'évaluation menée dans ce chapitre est fonctionnelle et qualitative. Elle vérifie si ProOWeb permet de générer une application web métier exploitable à partir de plusieurs processus d'une même organisation. Les critères retenus découlent directement des objectifs du chapitre précédent : exploiter BPMN et *spec-v1*, produire des artefacts applicatifs cohérents, préserver la traçabilité et permettre une évolution contrôlée.

La méthode croise l'inspection des artefacts produits et l'observation du comportement de l'application générée. Les fichiers BPMN conservés, les contrats `runtime`, les contrats `data`, le catalogue global et les classes générées côté backend

permettent de vérifier ce que ProOWeb a réellement produit. Les tests Cypress complètent cette lecture : ils ouvrent le *Process Runtime Workbench*, démarrent une instance de processus, complètent les tâches du parcours nominal et vérifient l'état final de l'instance.

**TABLE XXI –** *Critères retenus pour l'évaluation*

| Critère | Question posée | Mode d'observation |
|---|---|---|
| Complétude fonctionnelle | Les éléments nécessaires au runtime de processus sont-ils produits ? | Analyse des ressources générées, des contrats et des classes backend. |
| Cohérence BPMN-application | Les activités du modèle apparaissent-elles dans les contrats et les parcours testés ? | Comparaison entre BPMN, contrats `runtime` et tests Cypress. |
| Gestion des rôles | Les rôles de démarrage, de suivi et d'affectation sont-ils projetés ? | Analyse des contrats et des options de démarrage exposées au runtime. |
| Gestion des données | Les données partagées et les sorties d'activités sont-elles prises en compte ? | Vérification des contrats `data` et des entités partagées. |
| Capacité multi-processus | L'application regroupe-t-elle plusieurs processus dans un même environnement ? | Observation du catalogue, des ressources déployées et des tests par processus. |
| Évolution contrôlée | Les fichiers générés restent-ils identifiables après déploiement ? | Analyse du manifeste et des ressources suivies. |

### IV.5.2. Résultats observés

Les résultats observés se situent à deux niveaux. Au niveau du framework, ProOWeb lit les modèles, valide les informations, produit les contrats et écrit les artefacts du projet cible. Au niveau de l'application générée, les utilisateurs disposent d'un environnement de runtime dans lequel une instance de processus peut être créée, suivie et terminée selon les tâches exposées.

Le tableau suivant synthétise les observations réalisées sur le cas pratique. Il ne prétend pas mesurer toutes les qualités possibles d'un système d'information ; il montre ce que la mise en oeuvre permet déjà de vérifier par rapport aux objectifs du mémoire.

**TABLE XXII –** *Synthèse des résultats observés sur le cas d'étude*

| Élément évalué | Résultat observé | Interprétation |
|---|---|---|
| Ressources de processus | Les trois processus disposent chacun d'un BPMN, d'une spécification, d'un contrat runtime et d'un contrat data. | ProOWeb conserve le lien entre la source métier et les artefacts projetés. |
| Catalogue global | Les catalogues runtime et data regroupent les versions déployées. | L'application générée peut raisonner sur plusieurs processus dans un même socle. |
| Backend généré | Des classes telles que le registre de processus et les spécifications Java sont générées côté Spring Boot. | Les processus ne restent pas confinés à des fichiers de configuration. |
| Runtime frontend | Les tests utilisent le *Process Runtime Workbench* pour démarrer et terminer des instances. | Le frontend généré sait consommer les options de démarrage, les tâches et la progression d'une instance. |
| Tests e2e | Trois tests Cypress sont dédiés aux parcours de commande, de réapprovisionnement et de réclamation. | Chaque processus fait l'objet d'un parcours nominal vérifiable. |
| Manifeste | Les fichiers produits restent identifiés comme éléments gérés par ProOWeb. | La régénération peut être analysée au lieu d'être subie. |

**TABLE XXIII –** *Parcours vérifiés par les tests Cypress*

| **Processus testé** | **Parcours nominal vérifié** | **État attendu** |
|---|---|---|
| Commande client | Démarrage de l'instance, passage de commande, vérification des informations de commande | Instance terminée avec le marqueur `INSTANCE_COMPLETED`. |
| Réapprovisionnement du stock | Identification d'un produit en stock faible, création de la demande, examen de la demande | Instance terminée avec le marqueur `INSTANCE_COMPLETED`. |
| Réclamation de produit | Dépôt de la réclamation, qualification, vérification de la commande concernée | Instance terminée avec le marqueur `INSTANCE_COMPLETED`. |

### IV.5.3. Analyse de la capacité multi-processus

La capacité multi-processus occupe une place centrale dans cette évaluation, car elle touche directement à l'ambition de ProOWeb. Une application métier n'est pas seulement une suite d'écrans attachés à une procédure. Elle doit porter une partie de la vie de l'organisation, avec ses acteurs, ses données partagées et ses décisions qui se répondent d'un processus à l'autre.

Dans le cas d'étude, les trois processus ne sont donc pas juxtaposés au hasard. La commande client sollicite le stock ; le réapprovisionnement agit sur la disponibilité des produits ; la réclamation de produit se rattache à une commande déjà enregistrée et peut impliquer le service client, le service commercial ou le gestionnaire de stock. Cette proximité donne du sens au test réalisé : chaque processus conserve son propre déroulement, ses activités et ses transitions, tandis que l'application générée garde un socle commun pour les rôles, les tâches, l'historique, les produits et la supervision.

Ce résultat montre que ProOWeb peut soutenir une logique d'application organisationnelle. Les processus restent identifiables, mais ils ne sont pas enfermés dans trois applications séparées. L'utilisateur travaille dans un même environnement et l'application peut exposer les tâches selon les responsabilités déclarées dans les modèles et les spécifications.

### IV.5.4. Apports observés

L'évaluation met surtout en évidence la continuité obtenue entre la modélisation et l'application produite. Les activités décrites dans les modèles ne restent pas au niveau de la documentation : elles se retrouvent dans les tâches, les vues, les actions de traitement et les transitions du runtime. Cette continuité réduit le risque de voir le logiciel s'éloigner progressivement du processus métier qui l'a justifié.

*spec-v1* joue également un rôle important. Les rôles, les données manipulées, les règles de visibilité et les sorties d'activités ne sont pas ajoutés tardivement dans le code ; ils sont déclarés avant la génération et deviennent des éléments exploitables par ProOWeb. Le projet généré gagne ainsi en lisibilité, car les choix métier restent rattachés à une source identifiable.

Un autre apport tient à la structure du résultat obtenu. Le workspace généré n'est pas une accumulation de fichiers difficiles à relier. Il possède une organisation backend/frontend, des contrats de processus, des ressources de données, des scripts de vérification et un manifeste de fichiers gérés. Cette structuration facilite la reprise par un développeur et prépare les évolutions futures du projet.

### IV.5.5. Limites observées

ProOWeb présente aussi des limites. La qualité des modèles d'entrée reste déterminante : si le BPMN est ambigu ou si la spécification *spec-v1* est incomplète, la génération ne peut pas produire une application correcte. Le framework peut signaler des incohérences, mais il ne remplace pas l'analyse métier.

Certaines règles très spécifiques doivent également être traitées avec prudence. Dans une organisation de commerce électronique réelle, des décisions peuvent dépendre du transporteur, du prestataire de paiement, des promotions, des garanties commerciales ou des accords fournisseurs. Ces comportements nécessitent parfois une extension manuelle. L'évaluation menée ici garde elle aussi un périmètre limité : les tests Cypress vérifient les parcours du *Process Runtime Workbench* avec un backend simulé. Ils sont utiles pour contrôler l'enchaînement frontend/runtime, mais ils ne remplacent pas une campagne complète d'intégration, de charge, de sécurité et de persistance sur une infrastructure réelle.

Enfin, l'application générée reste un socle. Elle réduit la distance entre le modèle métier et le projet applicatif, mais elle ne supprime pas le rôle du développeur. Dans un projet réel, une équipe devra encore enrichir les tests, ajuster l'expérience utilisateur, raccorder les services externes et vérifier les règles métier propres à son environnement.

### IV.5.6. Discussion

Les résultats obtenus permettent de situer ProOWeb comme un framework de génération d'applications web métier pilotées par BPMN. Ils confirment, sur le cas e-commerce étudié, la logique défendue au chapitre précédent : BPMN décrit le déroulement métier, *spec-v1* complète les informations applicatives, les contrats intermédiaires stabilisent la génération et le manifeste prépare l'évolution. Sa valeur ne réside pas seulement dans la production de fichiers, mais dans la cohérence qu'il maintient entre processus, rôles, données, interfaces et suivi de génération.

Le cas pratique montre également que la spécification enrichie joue un rôle déterminant. BPMN décrit le déroulement du travail, mais l'application a besoin d'informations supplémentaires. En rendant ces informations explicites dans *spec-v1*, ProOWeb rend la génération plus contrôlable. Cet effort de spécification est le prix à payer pour produire une application métier plus complète qu'une simple maquette d'interface.

## IV.6. Conclusion

Ce chapitre a présenté la mise en oeuvre de ProOWeb et son application à un cas d'étude centré sur une organisation de commerce électronique. Il a montré comment les modèles BPMN enrichis sont importés, validés, transformés en contrats, puis déployés dans une application web métier générée.

Le cas pratique a permis d'observer ProOWeb sur trois processus appartenant au même contexte : commande client, réapprovisionnement du stock et réclamation de produit. L'évaluation confirme l'intérêt du framework pour produire une base applicative cohérente et multi-processus, tout en rappelant ses limites : qualité nécessaire des modèles d'entrée, effort de spécification, tests encore centrés sur des parcours nominaux et besoin d'évaluations complémentaires sur des environnements plus proches de la production.

# CONCLUSION GÉNÉRALE

SOMMAIRE

Bilan du travail . . . . . 74
Quelques perspectives . . . . . 75

## Bilan du travail

Ce mémoire s'est intéressé à la génération d'applications web métier à partir de modèles BPMN enrichis. Le problème traité vient du fait qu'un modèle de processus décrit bien le déroulement du travail, mais ne contient pas toujours toutes les informations nécessaires pour produire une application web réellement exploitable : rôles, données, formulaires, règles d'accès, suivi des instances et évolution du code généré.

Dans ce travail, nous avons proposé ProOWeb, un framework de génération d'applications web métier pilotées par BPMN. Pour atteindre cet objectif, nous avons d'abord étudié les notions de processus métier, de BPMN, d'applications web orientées processus et d'ingénierie dirigée par les modèles. Nous avons ensuite analysé les travaux existants afin de comprendre leurs apports et leurs limites lorsqu'il s'agit de passer d'un modèle de processus à une application complète.

La contribution proposée repose sur l'association d'un modèle BPMN et d'une spécification complémentaire, appelée *spec-v1*. Le modèle BPMN conserve le déroulement du processus, tandis que la spécification apporte les informations applicatives nécessaires à la génération. Ces éléments sont validés, organisés dans un modèle pivot, puis transformés en contrats et en artefacts logiciels.

Rendus au terme de ce mémoire, nous retenons principalement les contributions suivantes :

— la définition de ProOWeb comme framework de génération d'applications web métier à partir de modèles BPMN enrichis ;

— la proposition d'une spécification *spec-v1* permettant de compléter BPMN sans alourdir le diagramme de processus ;
— la mise en place d'une chaîne de génération allant des modèles d'entrée vers les contrats intermédiaires, puis vers les artefacts backend et frontend ;
— la prise en compte de la traçabilité et de la régénération contrôlée, afin que le projet généré puisse évoluer sans perdre le lien avec les modèles ;
— l'application du framework à une organisation de commerce électronique à travers trois processus : commande client, réapprovisionnement du stock et réclamation de produit.

Le cas d'étude a permis de montrer que ProOWeb ne se limite pas à la génération d'un processus isolé. Il peut produire une base applicative regroupant plusieurs processus appartenant à une même organisation. Les résultats obtenus restent toutefois à considérer comme une étape de validation. Le framework dépend encore de la qualité des modèles fournis, et certains aspects d'une application réelle, comme le paiement, la livraison, les intégrations externes ou les règles métier complexes, nécessitent encore des développements complémentaires.

## Quelques perspectives

Ce travail ouvre plusieurs pistes d'amélioration. Les plus importantes sont les suivantes :

— enrichir la spécification *spec-v1* pour prendre en compte des règles métier plus complexes et des validations plus fines ;
— développer un environnement d'assistance à la modélisation, afin d'aider l'utilisateur à construire et vérifier les modèles BPMN et les fichiers *spec-v1* ;
— étendre ProOWeb vers d'autres architectures techniques, afin de réduire sa dépendance à une pile technologique particulière ;
— renforcer la génération de tests automatiques liés aux transitions, aux rôles, aux formulaires et aux règles de visibilité ;
— évaluer le framework sur d'autres organisations et sur des processus plus nombreux, afin de mieux apprécier sa robustesse et sa portée.

En définitive, ProOWeb constitue une base de travail pour rapprocher la modélisation des processus métier et la réalisation d'applications web. Il ne remplace pas le développeur, mais lui offre un point de départ plus structuré, plus traçable et plus proche du fonctionnement réel de l'organisation.

# BIBLIOGRAPHIE


[1] M. Dumas, M. La Rosa, J. Mendling, and H. A. Reijers, *Fundamentals of Business Process Management*, 2nd ed. Berlin, Heidelberg : Springer, 2018.

[2] M. Weske, *Business Process Management : Concepts, Languages, Architectures*, 2nd ed. Berlin, Heidelberg : Springer, 2012.

[3] Object Management Group, "Business Process Model and Notation (BPMN), Version 2.0.2," https://www.omg.org/spec/BPMN/2.0.2/, Object Management Group, Tech. Rep., 2013, consulté le 22 mai 2026.

[4] ——, "Model Driven Architecture Guide, Revision 2.0," https://www.omg.org/cgi-bin/doc?ormsc/14-06-01, Object Management Group, Tech. Rep., 2014, guide officiel de la Model Driven Architecture.

[5] J.-M. Jézéquel, B. Combemale, and D. Vojtisek, *Ingénierie dirigée par les modèles : des concepts à la pratique*. Paris : Ellipses, 2012.

[6] E. Díaz, J. I. Panach, S. Rueda, and O. Pastor, "Towards a method to generate GUI prototypes from BPMN," in *2018 12th International Conference on Research Challenges in Information Science (RCIS)*. IEEE, 2018, pp. 1–12.

[7] A. Sajji, Y. Rhazali, and Y. Hadi, "A methodology for transforming bpmn to ifml into mda," *Bulletin of Electrical Engineering and Informatics*, vol. 11, no. 5, pp. 2773–2782, 2022.

[8] ——, "Graphical user interfaces generation from bpmn via ifml up to psm level," *International Journal of Advanced Computer Science and Applications*, vol. 14, no. 2, pp. 419–431, 2023.

[9] I. Drave, J. Michael, E. Müller, B. Rumpe, and S. Varga, "Model-driven engineering of process-aware information systems," *SN Computer Science*, vol. 3, no. 479, 2022.

[10] M. Hammer, "Reengineering work : Don't automate, obliterate," *Harvard Business Review*, 1990.

[11] W. M. P. van der Aalst, A. H. M. ter Hofstede, and M. Weske, *Business Process Management : A Survey*, ser. Lecture Notes in Computer Science. Berlin, Heidelberg : Springer, 2003, vol. 2678.

[12] M. Dumas, M. La Rosa, J. Mendling, and H. A. Reijers, *Fundamentals of Business Process Management*. Berlin : Springer, 2013, vol. 1.

[13] Bonitasoft, "L'essentiel du bpmn 2 : Comment réconcilier les besoins de la modélisation métier et informatique pour la gestion des processus métiers," Bonitasoft, Guide technique, s.d.

[14] P. Pichler, B. Weber, S. Zugal, J. Pinggera, J. Mendling, and H. A. Reijers, "Imperative versus declarative process modeling languages : An empirical investigation," in *Business Process Management Workshops*, ser. Lecture Notes in Business Information Processing, vol. 99. Springer, 2011, pp. 383–394.

[15] M. von Rosing, S. White, F. Cummins, and H. de Man, "Business process model and notation – bpmn," in *The Complete Business Process Handbook : Body of Knowledge from Process Modeling to BPM, Volume I*, M. von Rosing, H. von Scheel, and A.-W. Scheer, Eds. Waltham, MA : Morgan Kaufmann/Elsevier, 2015, pp. 429–453.

[16] S. Meghzili, "Ingénierie des logiciels : chapitre ingénierie dirigée par les modèles," 2022, support de cours.

[17] Object Management Group, "Meta object facility (mof)," https://www.omg.org/spec/MOF/, s.d., spécification officielle OMG.

[18] Auteur inconnu, "Ingénierie dirigée par les modèles (idm)," 2020, document de cours, chapitre 3.

[19] Object Management Group, "Mof query/view/transformation (qvt)," https://www.omg.org/spec/QVT/, s.d., spécification officielle OMG.

[20] I. Jacobs and N. Walsh, "Architecture of the world wide web, volume one," https://www.w3.org/TR/webarch/, World Wide Web Consortium, Tech. Rep., 2004, consulté le 22 mai 2026.

[21] R. T. Fielding, M. Nottingham, and J. Reschke, "HTTP Semantics," https://www.rfc-editor.org/rfc/rfc9110, Internet Engineering Task Force, Tech. Rep. RFC 9110, 2022, consulté le 22 mai 2026.

[22] MDN Web Docs, “How the web works,” https://developer.mozilla.org/en-US/docs/Learn_web_development/Getting_started/Web_standards/How_the_web_works, 2026, consulté le 22 mai 2026.

[23] A. Leff and J. T. Rayfield, “Web-application development using the model/view/controller design pattern,” in *Proceedings Fifth IEEE International Enterprise Distributed Object Computing Conference*. IEEE, 2001, pp. 118–127.

[24] MDN Web Docs, “Spa : Single-page application,” https://developer.mozilla.org/en-US/docs/Glossary/SPA, 2026, consulte le 09 juin 2026.

[25] R. T. Fielding, *Architectural styles and the design of network-based software architectures*. University of California, Irvine, 2000.

[26] E. Yuan and J. Tong, “Attributed based access control (abac) for web services,” in *Proceedings of the IEEE International Conference on Web Services (ICWS 2005)*. IEEE, 2005, pp. 561–569.

[27] Camunda, “Camunda Modeler and BPMN Process Orchestration Documentation,” https://docs.camunda.io/docs/components/modeler/about-modeler/, 2026, documentation officielle consultée en 2026.

[28] B. Kitchenham and S. Charters, “Guidelines for performing systematic literature reviews in software engineering,” Keele University and Durham University, Tech. Rep., 2007.

[29] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan *et al.*, “The prisma 2020 statement : an updated guideline for reporting systematic reviews,” *BMJ*, vol. 372, p. n71, 2021.

[30] W. M. P. van der Aalst, “Business process management : a comprehensive survey,” *ISRN Software Engineering*, vol. 2013, p. 507984, 2013.

[31] M. Dumas, M. La Rosa, J. Mendling, and H. A. Reijers, *Fundamentals of Business Process Management*, 2nd ed. Springer, 2018.

[32] Flowable, “Flowable Open Source BPMN Documentation,” https://www.flowable.com/open-source/docs/bpmn/ch02-GettingStarted/, 2026, documentation officielle consultée en 2026.

[33] jBPM, "jBPM Documentation : BPMN 2.0 Workflow Engine," https://www.jbpm.org/learn/documentation.html, 2026, documentation officielle consultée en 2026.

[34] Activiti, "Activiti BPMN Workflow Engine Documentation," https://www.activiti.org/userguide/, 2026, documentation officielle consultée en 2026.

[35] E. Schäffer, M. Schobert, T. Reichenstein, A. Selmaier, V. Stiehl, M. Herhoffer, M. Mala, and J. Franke, "Reference architecture and agile development method for a process-driven web platform based on the BPMN-standard and process engines," *Procedia CIRP*, vol. 103, pp. 146–151, 2021.

[36] Z. Hemel, R. Verhaaf, and E. Visser, "Webworkflow : An object-oriented workflow modeling language for web applications," 2008.

[37] U. T. Neto, T. Oliveira, R. Pillat, P. Alencar, D. Cowan, and G. Melo, "Akip process automation platform : A framework for the development of process-aware web applications," in *Proceedings of the International Conference on Enterprise Information Systems*, 2022.

[38] A. Sajji, Y. Rhazali, and Y. Hadi, "From interaction flow modeling language to symfony : an automated model transformation methodology," *Bulletin of Electrical Engineering and Informatics*, vol. 14, no. 5, pp. 3765–3777, 2025.

[39] N. Laaz, N. Kharmoum, and S. Mbarki, "Combining domain ontologies and bpmn models at the cim level to generate ifml models," in *Procedia Computer Science*, vol. 170, 2020, pp. 851–856.

[40] D. Pietruck and U. Steffens, "Generating REST APIs for workflow engines with process and data models," in *Proceedings of the 12th Central European Workshop on Services and their Composition*, ser. CEUR Workshop Proceedings, vol. 2825, 2020. [Online]. Available : https://ceur-ws.org/Vol-2825/paper3.pdf

[41] J. P. A. Hoyos and F. Restrepo-Calle, "Automatic source code generation for web-based process-oriented information systems," in *Proceedings of ENASE*, 2017.

[42] B. Uyanik and A. Sayar, "Developing web-based process management with automatic code generation," *Applied Sciences*, vol. 13, no. 21, p. 11737, 2023.

[43] E. Syriani, L. Luhunu, and H. Sahraoui, "Systematic mapping study of template-based code generation," 2018.

[44] A. Fedeli, F. Fornari, A. Polini, B. Re, V. Torres, and P. Valderas, "Flobp : a model-driven approach for developing and executing iot-enhanced business processes," *Software and Systems Modeling*, vol. 23, pp. 1217–1246, 2024.

[45] R. Salado-Cid, A. Vallecillo, K. Munir, and J. R. Romero, "Swel : A domain-specific language for modeling data-intensive workflows," *Business & Information Systems Engineering*, vol. 66, pp. 137–160, 2024.

[46] A. Delgado, D. Calegari, A. Arrigoni, A. Zunino, and R. Sosa, "Model-driven management of BPMN-based business process families," *Software and Systems Modeling*, 2022.

[47] T. Greifenberg, K. Hölldobler, C. Kolassa, M. Look, P. Mir Seyed Nazari, K. Müller, A. Navarro Pérez, D. Plotnikov, D. Reiss, A. Roth, B. Rumpe, M. Schindler, and A. Wortmann, "A comparison of mechanisms for integrating handwritten and generated code for object-oriented programming languages," in *Proceedings of the 3rd International Conference on Model-Driven Engineering and Software Development*, 2015, pp. 74–85.

[48] K. Rahad, O. Badreddin, and S. M. Reza, "The human in model-driven engineering loop : A case study on integrating handwritten code in model-driven engineering repositories," *Software : Practice and Experience*, vol. 51, no. 6, pp. 1308–1321, 2021.

[49] B. Dragaš, N. Todorović, T. Rajačić, and G. Milosavljević, "SeamlessMDD : Framework for seamless integration of generated and hand-written code," in *Web Engineering – 24th International Conference, ICWE 2024*. Springer, 2024, pp. 163–177.

[50] K. Rokis and M. Kirikova, "Exploring low-code development : A comprehensive literature review," *Complex Systems Informatics and Modeling Quarterly*, vol. 36, pp. 68–86, 2023.

[51] M. Pańkowska, "Low code development cycle investigation," 2024.

[52] A. Bucaioni, A. Cicchetti, and F. Ciccozzi, "Modelling in low-code development : a multi-vocal systematic review," *Software and Systems Modeling*, vol. 21, pp. 1959–1981, 2022.

[53] D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi, and M. Wimmer, "Low-code development and model-driven engineering : Two sides of the same coin ?" *Software and Systems Modeling*, vol. 21, pp. 437–446, 2022.

[54] M. R. Krouwel, M. Op 't Land, and H. A. Proper, "From enterprise models to low-code applications : mapping DEMO to mendix ; illustrated in the social housing domain," *Software and Systems Modeling*, vol. 23, no. 4, pp. 837–864, 2024.

[55] F. Martínez-Lasaca, P. Díez, E. Guerra, and J. de Lara, "A model and workflow-driven approach for engineering domain-specific low-code platforms and applications," *Software and Systems Modeling*, vol. 25, pp. 579–614, 2026.

[56] B. A. Zemtsop Ndadji, "Vers une approche basée sur l'ingénierie dirigée par les modèles, de gestion des workflows dans une application web," Mémoire de Master en informatique, Université de Dschang, Dschang, Cameroun, 2022.

[57] M. D. Nguedia Momo, "Vers une approche grammaticale de gestion des processus opérationnels dans un réseau IoT," Mémoire de Master en informatique, Université de Dschang, Dschang, Cameroun, 2022.